

Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory

ERIC GIOVANNINI, University of Michigan, USA

TINGTING DING, University of Michigan, USA

MAX S. NEW, University of Michigan, USA

Gradually typed programming languages, which allow for soundly mixing static and dynamically typed programming styles, present a strong challenge for metatheorists. Even the simplest sound gradually typed languages feature at least recursion and errors, with realistic languages featuring furthermore runtime allocation of memory locations and dynamic type tags. Further, the desired metatheoretic properties of gradually typed languages have become increasingly sophisticated: validity of ~~type-based~~ type-based equational reasoning as well as the relational property known as graduality. Many recent works have tackled verifying these properties, but the resulting mathematical developments are highly repetitive and tedious, with few reusable theorems persisting across different developments.

In this work, we present a new denotational semantics for gradual typing developed using guarded domain theory. Guarded domain theory combines the generality of step-indexed logical relations for modeling advanced programming features with the modularity and reusability of denotational semantics. We demonstrate the feasibility of this approach with a model of a simple gradually typed lambda calculus and prove the validity of beta-eta equality and the graduality theorem for the denotational model. This model should provide the basis for a reusable mathematical theory of gradually typed program semantics. Finally, ~~a recent extension to Agda allows synthetic guarded domain theory to be readily mechanized in a proof assistant, and we formalize many we have mechanized most~~ of the core theorems of our development in Guarded Cubical Agda, a recent extension of Agda with support for the guarded recursive constructions we use.

ACM Reference Format:

Eric Giovannini, Tingting Ding, and Max S. New. 2018. Denotational Semantics of Gradual Typing using Synthetic Guarded Domain Theory. In *Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY*. ACM, New York, NY, USA, ?? pages. <https://doi.org/10.1145/1122445.1122456>

1 INTRODUCTION

1.1 Gradual Typing and Graduality

In programming language design, there is a tension between *static* typing and *dynamic* typing disciplines. With static typing, the code is type-checked at compile time, while in dynamic typing, the type checking is deferred to run-time. Both approaches have benefits ~~and excel~~ in different scenarios, with static typing ~~the programmer is assured that if the program passes the type checker, their program is free of type errors, and moreover, soundness of the equational theory implies that program optimizations are valid. Meanwhile, dynamic typing allows the programmer~~

Authors' addresses: Eric Giovannini, Electrical Engineering and Computer Science, University of Michigan, USA, ericgio@umich.edu; Tingting Ding, Electrical Engineering and Computer Science, University of Michigan, USA, tingtind@umich.edu; Max S. New, Electrical Engineering and Computer Science, University of Michigan, USA, maxsnew@umich.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Woodstock '18, June 03–05, 2018, Woodstock, NY

© 2018 ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/10.1145/1122445.1122456>

to rapidly prototype their application code without needing to offer compile-time assurance of a program's type safety and type-based reasoning principles that justify program optimizations, and dynamic typing allowing for rapid prototyping of a codebase without committing to fixed type signatures for their functions. Most languages choose between static or dynamic typing and as a result, programmers that initially write their code in a dynamically typed language in order to benefit from faster prototyping and development time need to rewrite some or all of their codebase in a static language if they would like to receive the benefits of static typing once their codebase has matured.

Gradually typed Gradually typed languages [33, 37] seek to resolve this tension by allowing for both static and dynamic typing disciplines to be used in the same codebase, and by supporting. These languages support smooth interoperability between statically-typed and dynamically-typed code. This flexibility allows programmers to begin their projects in a dynamic style and enjoy the benefits of dynamic typing related to rapid prototyping and easy modification while the codebase "solidifies". Over time, as parts of the code become more mature and the programmer is more certain of what the types should be, the code can be styles, allowing the programmer to begin with fully dynamically-typed code and gradually migrated migrate portions of the codebase to a statically typed style without needing to rewrite the project in a completely different language.

In order for this to work as expected, gradually typed languages should allow for different parts of the codebase to be in different places along the spectrum from dynamic to static, and allow for those different parts to interact with one another. Moreover, gradually typed languages should support the smooth migration from dynamic typing to static typing, in that the programmer can initially leave off the typing annotations and provide them later without altering the meaning of the program. Furthermore, the parts of the program that are written in a dynamic style should soundly interoperate with the parts that are written in a static style. That is, the interaction between the static and dynamic components of the codebase should preserve, to the extent possible, the guarantees made by the static types. In particular, while statically typed code can error at runtime in a gradually typed language, such an error can always be traced back to a dynamically typed term that violated the typing contract imposed by statically typed code. Further, static type assertions are sound in the static portion, and should enable type-based reasoning and optimization.

One of the fundamental theorems for gradually typed languages is *graduality*, also known as the *dynamic gradual guarantee*, originally defined by Siek, Vitousek, Cimini, and Boyland [26, 34]. Informally, graduality says that going from a migrating code from dynamic to static style typing should only allow for the introduction of static or dynamic type errors, and not otherwise change the behavior of the program. This is a way to capture programmer intuition that increasing type precision corresponds to a generalized form of runtime assertions in that there are no observable behavioral changes up to the point of the first dynamic type error¹. Fundamentally, this property comes down to the behavior of *runtime type casts*, which implement these generalized runtime assertions.

Additionally, gradually typed languages should offer some of the benefits of static typing. While classical standard type soundness, that well-typed programs are free from runtime errors, is not compatible with runtime type errors, we can instead focus on it is possible instead to prove that gradually typed languages validate type-based reasoning. For instance, while dynamically typed λ calculi only satisfy β equality for their type formers, statically typed λ calculi additionally satisfy type-dependent η properties that ensure that functions are determined by their behavior under application and that pattern matching on data types is safe and exhaustive. A gradually

¹once a dynamic type error is raised, in languages where the type error can be caught, program behavior may then further diverge, but this is typically not modeled in gradual calculi.

typed calculus that validates these type-dependent η laws then provides some of the type-based reasoning that dynamic languages lack.

More concretely, consider a gradually typed language whose only effects are gradual type errors and divergence. Then if we fix a result type of natural numbers, a whole program semantics is a partial function from closed programs to either natural numbers or errors:-

$$- \Downarrow: \{M \mid \vdash M : \text{Nat}\} \rightarrow \mathbb{N} \cup \{\mathcal{U}\}$$

where $\mathcal{U}$ is notation for a runtime type error. We write $M \Downarrow n$ and $M \Downarrow \mathcal{U}$ to mean this semantics is defined as a number or error, and $M \Uparrow$ to mean the semantics is undefined, representing divergence. A well-behaved semantics should then satisfy several properties. First, it should be *adequate*: natural number constants should step to themselves. Second it should validate type based reasoning. To formalize type based reasoning we give a typed equational theory for terms of the language $M \equiv N$ for when two terms should be considered equivalent. Then we want to verify that the big step semantics respects this equational theory: if closed programs $M \equiv N$ are equivalent in the equational theory then they have the same semantics, $M \Downarrow n \iff N \Downarrow n$ and $M \Uparrow \iff N \Uparrow$ and $M \Downarrow \mathcal{U} \iff N \Downarrow \mathcal{U}$. Lastly, the graduality property is defined by giving an *inequational* theory called term precision, where $M \sqsubseteq N$ roughly means that M and N have the same type erasure and M has at each point in the program a more precise/static type than N . Then, the graduality property states that if $M \sqsubseteq N$ are whole programs then M must either have the same behavior as N or error: Either $M \Downarrow \mathcal{U}$ or $M \Downarrow n$ and $N \Downarrow n$ or $M \Uparrow$ and $N \Uparrow$ ².

1.2 Denotational Semantics in Guarded Domain Theory

Our goal in this work is to provide an *expressive, reusable, compositional* semantic framework for defining such well-behaved semantics of gradually typed programs. Our approach to achieving this goal is to provide a compositional *denotational semantics*, mapping types to a kind of semantic domain, terms to functions and relations such as term precision to proofs of semantic relations between the denoted functions. Since the denotational constructions are all syntax-independent, the constructions we provide may be reused for similar languages. Since it is compositional, components can be mixed and matched depending on what source language features are present. **Providing this**

Providing a semantics for gradual typing is inherently complicated in that it involves: (1) recursion and recursive types through the presence of dynamic types, (2) effects in the form of divergence and errors (3) relational models in capturing the graduality property. Recursion and recursive types must be handled using some flavor of domain theory. Effects can be modeled using monads in the style of Moggi, or adjunctions in the style of Levy[18, 19, 22, 23]. Relational properties and their verification lead naturally to the use of reflexive graph categories or double categories[11, 29].

The only prior denotational semantics for gradual typing was given by New and Licata and is based on a classical Scott-style *domain theory* [29]. The fundamental idea is to equip ω -CPOs with an additional “error ordering” $\sqsubseteq$ which models the graduality ordering, and for casts to arise from *embedding-projection pairs*. Then the graduality property follows as long as all language constructs can be interpreted using constructions that are monotone with respect to the error ordering. This framework has the benefit of being compositional, and was expressive enough to be extended to model dependently typed gradual typing [?] [17]. However, an approach based on classical domain theory has fundamental limitations: domain theory is appears incapable of modeling certain perversely recursive features of programming languages such as dynamic type tag generation and

²we use a slightly more complex definition of this relation in our technical development below that is classically equivalent but constructively weaker

higher-order references [7], which are commonplace in real-world gradually typed systems as well as gradual calculi. Our long-term goal is to develop a semantics that will denotational approach that can scale up to these advanced features, and so we begin with this work by departing from must abandon classical domain theory as the foundation in order to make progress. In this work, we provide first steps towards this goal by adapting work on a simple gradually typed lambda calculus to guarded domain theory, a task that already involves nontrivial issues. Our goal is for this model to scale to dynamic type tag generation and higher-order references in future work.

The Guarded domain theory is the main denotational alternative to classical domain theory that can successfully model these advanced features is guarded domain theory, which we adopt in this work. While classical domain theory is based on modeling types as ordered sets with certain joins, guarded domain theory is based on an entirely different foundations, sometimes (ultra)metric spaces but more commonly as “step-indexed sets”, i.e., objects in the *topos of trees* i.e., [6]. Such an object consists of a family $\{X_n\}_{n \in \mathbb{N}}$ of sets along with restriction functions $r_n : X_{n+1} \rightarrow X_n$ for all n . (in category theoretic terminology, these are presheaves on the poset of natural numbers [6].) We think of such a $\mathbb{N}$ -indexed set as modeling a kind of “sequence of approximations” an infinite sequence of increasingly precise approximations to the true type being modeled. Key to guarded domain theory is that there is a “later” an operator $\triangleright$ on types. Thinking of types as a sequence step-indexed sets called “later”. In terms of sequences of approximations, the later operator delays the approximation by one step. Then the crucial axiom of guarded domain theory is that any guarded domain equation $X \cong F(\triangleright X)$ has a unique solution. This allows guarded domain theory to model essentially *any* recursive concept, with the caveat that it is guarded by a later. This caveat is the main source of difficulty in adapting a semantic approach based on classical domain theory to guarded domain theory: classical domain theory has limitations in what it can model, but it provides exact solutions to domain equations when it applies. When adapting the New-Licata approach to guarded domain theory, this means we must work with an intensional semantics, one where unfolding the dynamic type requires an observable computational step: e.g., a function that pattern matches on an element of the dynamic type and then returns it unchanged is not equal to the identity function, because it “costs” a step to perform the pattern match. This leads to the main departure of our semantics from the New-Licata approach: while the New-Licata semantics equips ω -CPOs with an error ordering, we work with types equipped with not just an error ordering but also a bisimilarity relation that reflects when elements differ only in the number of abstract computational steps that they take. Because bisimilarity, unlike equality, is not transitive this necessitates further changes to the core concepts of our semantics: the recursion is guarded by a use of the later operator.

1.3 Synthetic Guarded Domain Theory

While guarded domain theory can be presented analytically using ultrametric spaces or the topos of trees, in practice it is considerably simpler to work *synthetically* by working in a non-standard foundation such as foundational system such as guarded type theory. In guarded type theory where the later modality later is taken as a primitive operation on types, and we take as an axiom that guarded domain equations have a (necessarily unique) solution. This has the added benefit that providing a denotational semantics of gradually typed terms is as simple as a semantics of an effectful language using a monad on the category The benefit of sets this synthetic approach is that when working in the non-standard foundation, we don’t need to model an object language type as a step-indexed set, but instead simply as a set, and object-language terms can be modeled in the Kleisli category of a simple monad defined using guarded recursion. Not only does this make on-paper reasoning about guarded domain theory easier, it also enables a simpler avenue to verification in a proof assistant. Whereas formalizing analytic guarded domain theory would require a significant theory of presheaves and making sure that all constructions are functors on

categories of presheaves, formalizing synthetic guarded domain theory can be done by directly adding the later modality and the guarded fixed point property axiomatically. In this paper, we work informally in a guarded type theory which is described in more detail in Section 3.1.

~~Because the solutions we obtain working in guarded domain theory are constructed as a sequence of increasingly better approximations, when we want to establish a property that holds in the limit, we must reason about all finite approximations. In the setting of the topos of trees, this manifests as the need to consider global elements of~~

1.3 Adapting the New-Licata Model to the Guarded Setting

Since guarded domain theory only provides solutions to guarded domain equations, there is no systematic way to convert a classical domain-theoretic semantics to a guarded one. Classical domain theory has limitations in what it can model, but it provides exact solutions to domain equations when it applies. When adapting the New-Licata approach to guarded domain theory, the presheaves, i.e., a family of elements $x_i \in X_i$ compatible with the restriction maps, presence of later in the semantics makes it intensional: unfolding the dynamic type requires an observable computational step. For example, ~~consider the set of programs that may take a computational step or return unit. In the topos of trees, we can model this set as the indexed family of sets $X_n = \{0, 1, \dots, n-1, \perp\}$ for $n \geq 1$, where i denotes a program that steps i times and then returns, and $\perp$ denotes a program that fails to terminate in $n-1$ steps or fewer. For any fixed n , the set X_n fails to distinguish a program that takes n steps and then terminates from a program that never terminates. On the other hand, if we instead consider the denotation of a program to be a global element of a function that pattern matches on an element of the dynamic type and then returns it unchanged is not equal to the identity function, because it “costs” a step to perform the pattern match.~~

The intensional nature of guarded domain theory leads to the main departure of our semantics from the New-Licata approach. Because casts involve computational steps, the presheaf $\{X_n\}$ —a family of elements $x_i \in X_i$ for all $i \in \mathbb{N}$ —then the denotation of the diverging program will be distinct from that of a program that terminates, regardless of the number of steps it takes. Graduality property must be insensitive to the steps taken by terms. This means that the model must allow for reasoning up to weak bisimilarity, where two terms are weakly bisimilar if they differ only in their number of computational steps. However, the weak bisimilarity relation is not transitive in the guarded setting, which follows from a no-go theorem we establish (Theorem 4.1). The New-Licata proofs freely use transitivity, and we argue in Section 4 that some amount of transitive reasoning is necessary for defining a syntax-independent model of gradual typing. As a result, the lack of transitivity of weak bisimilarity presents a challenge in adapting the New-Licata model to the guarded setting. Our solution is to work with a combination of weak bisimilarity and step-sensitive reasoning, where the latter notion is transitive. To deal with the fact that casts take computational steps, we introduce the novel concept of syntactic perturbations, which are explicit synchronizations that we manipulate syntactically. The combination of lock-step reasoning with perturbations is transitive and is able to account for the step-insensitive nature of the graduality property.

~~In the setting of synthetic guarded domain theory, this same idea applies. To construct a global solution we must use an additional construct,~~

1.4 Adequacy

Having defined the model of gradual typing in guarded type theory, we need to show that the guarded model induces a sensible set-theoretic semantics in ordinary mathematics. We cannot hope to derive such a semantics for all types (e.g., the $\Box$ -modality $[?]$, whose semantics in the topos

of trees model is to compute the global elements of a presheaf. A related approach involves objects known as *clocks*, whereby *clock quantification* [3] provides a means to obtain a global solution (dynamic type), but for the subset of closed terms of base type, we should be able to extract a well-behaved semantics for which the graduality property holds.

This has important ramifications for defining an adequate denotational semantics. More concretely, consider a gradually typed language whose only effects are gradual type errors and divergence (errors arise from failing casts; divergence arises because we can use the dynamic type to encode untyped lambda calculus; see Section 2). If we fix a result type of natural numbers, a “big-step” semantics is a partial function from closed programs to either natural numbers or errors:

$$- \Downarrow : \{M \mid \vdash M : \text{Nat}\} \rightarrow \mathbb{N} \cup \{\mathcal{U}\}$$

where $\mathcal{U}$ is notation for a runtime type error. We write $M \Downarrow n$ and $M \Downarrow \mathcal{U}$ to mean this semantics is defined as a number or error, and $M \Uparrow$ to mean the semantics is undefined, representing divergence. A well-behaved semantics should then satisfy several properties. First, it should be *adequate*: natural number constants should evaluate to themselves: $n \Downarrow n$. Second, it should validate type based reasoning. To formalize type based reasoning, languages typically have an equational theory $M \equiv N$ specifying when two terms should be considered equivalent. Then we want to verify that the big step semantics respects this equational theory: if closed programs $M \equiv N$ are equivalent in the equational theory then they have the same semantics, $M \Downarrow n \iff N \Downarrow n$ and $M \Uparrow \iff N \Uparrow$ and $M \Downarrow \mathcal{U} \iff N \Downarrow \mathcal{U}$.

We have to take care when defining such a big-step semantics in guarded domain type theory, as we seek to do in this paper: we do not want our denotational semantics to conflate a diverging program with a program that fails to terminate in sufficiently few steps. Thus, establishing adequacy of a semantics in guarded domain theory will require a means of globalizing, a point we will return to in Section 3.5. A naive definition of termination is incorrect: we can prove using guarded recursion that an infinite loop terminates! This is because the definition of termination inside guarded logic has a different meaning, essentially meaning that for every finite number n , either the program terminates in $< n$ steps or it takes $n + 1$ steps, which is equivalent to a statement of progress rather than termination. For this reason, we need a kind of “escape hatch” in our ambient type theory to leave the guarded setting and make definitions that are interpreted as constructions in ordinary mathematics. We incorporate this by using *clock quantification* [3, 5] in our type theory. In particular, the guarded type theory in which we work has a notion of clocks that index all of our guarded constructions. This is discussed in more detail in Section 3.5.

To define the big-step semantics, we apply clock quantification to obtain an interpretation of closed programs of base type as partial functions. We describe the process in more detail in Section 3.5. The resulting term semantics is adequate, and furthermore it validates the equational theory, since equivalent terms in the equational theory denote equal terms in the big-step semantics.

Lastly, the semantics should be adequate for the graduality property. Graduality is typically axiomatized by giving an *inequational* theory called term precision, where $M \sqsubseteq N$ roughly means that M and N have the same type erasure and M has at each point in the program a more precise/static type than N . Then adequacy says that if M and N are whole programs returning type Nat and $M \sqsubseteq N$, then the denotations of M and N as big-step terms should be related in the expected way: either M errors, or M and N have the same extensional behavior. That is, either $M \Downarrow \mathcal{U}$ or $M \Downarrow n$ and $N \Downarrow n$ or $M \Uparrow$ and $N \Uparrow$ ². To prove that the semantics is adequate for graduality, we again apply clock quantification, this time to the relation that denotes the term precision ordering. We show

²we use a slightly more complex definition of this relation in our technical development below that is classically equivalent but constructively weaker

that a term precision ordering $M \sqsubseteq N$ implies the corresponding ordering on the partial functions denoted by M and N . The details of the proof are given in Section 5.4.

1.5 Contributions and Outline

The main contribution of this work is a compositional denotational semantics ~~for gradually typed languages in guarded type theory for a simple gradually typed language~~ that validates $\beta\eta$ equality and satisfies a graduality theorem. ~~A great deal of the~~ Within our semantics, the notion of syntactic perturbation is our most significant technical contribution, allowing us to successfully adapt the denotational approach of New and Licata to the guarded setting. Most of our work has further been verified in Guarded Cubical Agda [39], demonstrating that the semantics is readily mechanizable. We provide an overview of the mechanization effort, and what remains to be formalized, in Section 6.3.

- (1) ~~First, we give a simple concrete term semantics where we show how to model the dynamic type as a solution to a guarded domain equation.~~
- (2) ~~Next, we identify where prior work on classical domain theoretic semantics of gradual typing breaks down when using guarded semantics of recursive types.~~
- (3) ~~We develop a key new concept of syntactic perturbations, which allow us to recover enough extensional reasoning to model the graduality property compositionally.~~
- (4) ~~We combine this insight together with an abstract categorical model of gradual typing using reflexive graph categories and call-by-push-value to give a compositional construction of our denotational model.~~
- (5) ~~We prove that the resulting denotational model provides a well behaved semantics as defined above by proving adequacy, respect for an equational theory and the graduality property.~~

The paper is laid out as follows:

- (1) In Section 2 we fix our input language, a fairly typical gradually typed cast calculus.
- (2) In Section 3 we develop a ~~first~~ denotational semantics in synthetic guarded domain theory ~~that satisfies adequacy and respects for the terms of the gradual lambda calculus. The model is adequate and validates~~ the equational theory, but ~~does not validate it does not satisfy~~ graduality. We use this to introduce some of our main technical tools: modeling recursive types in guarded type theory and modeling effects using call-by-push-value.
- (3) In Section 4 we show where the New-Licata classical domain theoretic approach fails to adapt cleanly to the guarded setting and explore the difficulties of proving graduality in an intensional model. ~~We prove a no-go theorem about extensional, transitive relations, introduce the lock-step error ordering and weak bisimilarity relation, and motivate the need for perturbations.~~
- (4) In Section 5 we describe the ~~resulting construction of the~~ model in detail. ~~We then, and discuss the translation of the syntax and axioms of the gradually typed cast calculus into the model. Lastly, we prove that the resulting denotational model is adequate model is adequate~~ for the graduality property: ~~a closed term precision $M \sqsubseteq N$: Nat has the expected semantics, i.e., that M errors or M and N have the same extensional behavior.~~
- (5) In Section 6 we discuss prior work on proving graduality, ~~our the~~ partial mechanization of ~~these results and discuss our results in Agda, and~~ future directions for denotational semantics of gradual typing.

2 SYNTACTIC THEORY OF GRADUALLY TYPED LAMBDA CALCULUS

Here we give an overview of a fairly standard cast calculus for gradual typing along with its (in-)equational theory that capture our desired notion of type-based reasoning and graduality. The main departure from prior work is our explicit treatment of type precision derivations and an equational theory of those derivations.

We give the basic syntax and select typing rules in Figure 1. We include a dynamic type, a type of numbers, the call-by-value function type $A \rightarrow A'$ and products. We include a syntax for *type precision* derivations $c : A \sqsubseteq A'$; the typing is given in Figure 1. Any type precision derivation $c : A \sqsubseteq A'$ induces a pair of casts, the upcast $\text{up } c : A \rightarrow A'$ and the downcast $\text{dn } c : A' \rightarrow A$. The syntactic intuition is that c is a proof that A is “less dynamic” than A' . Semantically, this gives us coercions back and forth where the upcast is (to a first-order) a pure function whereas the downcast can fail. These casts are inserted automatically in an elaboration from a surface language. In this work, we are focused on semantic aspects and so elide these standard details. The syntax of precision derivations includes reflexivity $r(A)$ and transitivity cc' as well as monotonicity $c \rightarrow c'$ and $c \times c'$ that are *covariant* in all arguments and finally generators $\text{Inj}_{\mathbb{N}}, \text{Inj}_{\rightarrow}, \text{Inj}_{\times}$ that correspond to the type tags of our dynamic type. We additionally impose an equational theory $c \equiv c'$ on the derivations that implies that the corresponding casts are weakly bisimilar in the semantics. We impose category axioms for the reflexivity and transitivity and functoriality for the monotonicity rules. We note the following two admissible principles: any two derivations $c, c' : A \sqsubseteq A'$ of the same fact are equivalent $c \equiv c'$ and for any A , there is a derivation $\text{dyn}(A) : A \sqsubseteq ?$. That is, $?$ is the “most dynamic” type.

There is a more common set of rules for type precision where reflexivity and transitivity are admissible, and whenever $A \sqsubseteq A'$, there is a unique precision derivation witnessing this. These rules are shown in Section A in the Appendix. The reason for choosing our system rather than that one is that in our semantics, equivalent type precision derivations do not denote *equal* relations. Instead, they denote relations that are *quasi-equivalent*, i.e., if two terms are related by one then they are related also by the other up to insertion of delays (see Definition 5.8 for the details). However, because all type precision derivations in our system are equivalent, it is straightforward to define a translation from the more standard system of type and term precision into ours, so ultimately our graduality proofs can still be applied to the standard formulation.

Next, we consider the axiomatic (in-)equational reasoning principles for terms: $\beta\eta$ equality and term precision in Figure 2. We include standard CBV $\beta\eta$ rules for function and product types, as well as equations stating that casts are given functorially. Next, we have *term* precision, an extension of type precision to terms. The form of the term precision rule is $\Delta \vdash M \sqsubseteq M' : c$ where Δ is a context where variables are assigned to type precision derivations. The judgment is only well formed when every use of $x : c$ for $c : A \sqsubseteq A'$ is used with type A in M and A' in M' and similarly the output types match c . We elide the congruence rules for every type constructor, e.g., that $M \sqsubseteq M'$ and $N \sqsubseteq N'$ that $MN \sqsubseteq M'N'$. With such congruence rules, reflexivity $M \sqsubseteq M$ is admissible. Transitivity, on the other hand, is intentionally not taken as a primitive rule, matching the original formulation of the dynamic gradual guarantee [34]. We include a rule that says that equivalent type precision derivations $c \equiv c'$ are equivalent for the purposes of term precision. Finally, we include 4 rules for reasoning about casts. Intuitively these say that the upcast is a kind of *least upper bound* and dually that the downcast is a *greatest lower bound*.

As a higher-order gradually typed language, we inherently have to deal with two effects: errors and divergence. Errors arise from failing casts, e.g. casting a number to dynamic to a function $\text{dn Inj}_{\rightarrow}, \text{up Inj}_{\mathbb{N}} x$. Divergence arises because our dynamic type allows us to encode

Types A	$::=$	$\text{Nat} \mid ? \mid A \multimap A' \mid A \times A'$
Type Precision c	$::=$	$r(A) \mid cc' \mid \text{Inj}_{\rightarrow} \mid \text{Inj}_{\mathbb{N}} \mid \text{Inj}_{\times} \mid c \multimap c' \mid c \times c'$
Values V	$::=$	$x \mid \text{up } c V \mid \text{zro} \mid \text{suc } V \mid \lambda x.M \mid (V, V')$
Terms M, N	$::=$	$\mathbb{U} \mid \text{up } c M \mid \text{dn } c M \mid \text{zro} \mid \text{suc } M \mid \lambda x.M$ $\mid MN \mid (M, N) \mid \text{let } (x, y) = M \text{ in } N$
Contexts Γ	$::=$	$\cdot \mid \Gamma, x : A$
Ctx Precision Δ	$::=$	$\cdot \mid \Delta, x : c$

$\Gamma \vdash M : A$	$c : A \sqsubseteq A'$	$\Gamma \vdash N : A'$	$c : A \sqsubseteq A'$	$\Gamma \vdash \mathbb{U} : A$
$\Gamma \vdash \text{up } c M : A'$		$\Gamma \vdash \text{dn } c N : A$		

$r(A) : A \sqsubseteq A$	$\frac{c : A \sqsubseteq A' \quad c' : A' \sqsubseteq A''}{cc' : A \sqsubseteq A''}$	$\text{Inj}_{\rightarrow} : ? \multimap ? \sqsubseteq ?$	$\text{Inj}_{\mathbb{N}} : \text{Nat} \sqsubseteq ?$
--------------------------	--	--	--

$\text{Inj}_{\times} : ? \times ? \sqsubseteq ?$	$\frac{c_i : A_i \sqsubseteq A'_i \quad c_o : A_o \sqsubseteq A'_o}{c_i \multimap c_o : (A_i \multimap A_o) \sqsubseteq (A'_i \multimap A'_o)}$	$\frac{c_1 : A_1 \sqsubseteq A'_1 \quad c_2 : A_2 \sqsubseteq A'_2}{c_1 \times c_2 : (A_1 \times A_2) \sqsubseteq (A'_1 \times A'_2)}$
--	---	--

$r(A)c \equiv c$	$c \equiv cr(A')$	$c(c'c'') \equiv (cc')c''$	$r(A_i \multimap A_o) \equiv r(A_i) \multimap r(A_o)$
$r(A_1 \times A_2) \equiv r(A_1) \times r(A_2)$	$(c_i \multimap c_o)(c'_i \multimap c'_o) \equiv (c_i c'_i \multimap c_o c'_o)$		
$(c_1 \times c_2)(c'_1 \times c'_2) \equiv (c_1 c'_1 \times c_2 c'_2)$			

Fig. 1. GTLC Cast Calculus Syntax, Type Precision Derivations and Precision Equivalence

untyped lambda calculus, and so we can encode the Ω term with the help of casts $\Omega = (\lambda x : ? . (\text{dn } \text{Inj}_{\rightarrow} x) x) (\text{up } \text{Inj}_{\rightarrow} (\lambda x : ? . (\text{dn } \text{Inj}_{\rightarrow} x) x))$.

Our goal in the remainder of this work is to develop compositional denotational semantics of types, terms, type and term precision from which we can easily extract a big step semantics that satisfies graduality and respects the equational theory of the calculus.

3 A DENOTATIONAL SEMANTICS FOR TYPES AND TERMS

As a first step to giving a well-behaved semantics of GTLC, in this section we define a simpler semantics that validates all of our desired properties except for graduality: that is, it is adequate and validates $\beta\eta$ equality. Our relational semantics will then be a refinement of this simpler design. In the process we will introduce some of the technical tools we will use in the refined semantics: guarded type theory and call-by-push-value. While these tools can be somewhat technical we emphasize that due to the use of synthetic guarded domain theory, this initial semantics is essentially just a “naïve” denotational semantics of an effectful language arising from a monad on sets. But since we are working in the non-standard foundation of guarded type theory our notion of sets allows for the construction of more sophisticated sets and monads than classical logic allows.

3.1 Guarded Type Theory

We begin with a brief overview of the background theory that we use in this work. *Synthetic guarded domain theory*, or SGDT for short, is an axiomatic framework in which we can reason about non-well-founded recursive constructions while abstracting away the specific details of step-indexing that we would need to track if we were working analytically. This allows us to avoid

$$\begin{array}{l}
(442) \quad (\lambda x.M)(V) = M[V/x] \qquad (V : A \multimap A') = \lambda x.V x \\
(443) \\
(444) \quad \text{let } (x, y) = (V, V') \text{ in } N = N[V/x, V'/y] \qquad M[V : A \times A'/p] = \text{let } (x, y) = V \text{ in } M[(x, y)/p] \\
(445) \\
(446) \quad \underline{\text{up } (r(A)) M = \text{Mup } c' \text{ up } c M = \text{up } cc' M \text{dn } (r(A)) M = M \text{dn } c \text{dn } c' M = \text{dn } cc' M}} \\
(447) \\
(448) \\
(449) \\
(450) \quad \frac{}{\Delta \vdash M \sqsubseteq M' : c} \text{c} \equiv c' \quad \text{EQUIVTyPREC} \quad \Delta \vdash M \sqsubseteq M' : c' \qquad \text{ERRBOT} \quad \Delta \vdash \mathcal{U} \sqsubseteq M : c \\
(451) \\
(452) \\
(453) \\
(454) \quad \frac{}{M \sqsubseteq M' : cc_r} \text{UpL} \quad \text{up } c M \sqsubseteq M' : c_r \qquad \frac{}{M \sqsubseteq M' : c_l} \text{UpR} \quad M \sqsubseteq \text{up } c M' : c_l c \\
(455) \\
(456) \\
(457) \\
(458) \\
(459) \quad \frac{}{M \sqsubseteq M' : c_r} \text{DnL} \quad \text{dn } c M \sqsubseteq M' : cc_r \qquad \frac{}{M \sqsubseteq M' : c_l c} \text{DnR} \quad M \sqsubseteq \text{dn } c M' : c_l \\
(460) \\
(461) \\
(462) \\
(463) \\
(464) \\
(465) \\
(466) \\
(467) \\
(468) \\
(469) \\
(470) \\
(471) \\
(472) \\
(473) \\
(474) \\
(475) \\
(476) \\
(477) \\
(478) \\
(479) \\
(480) \\
(481) \\
(482) \\
(483) \\
(484) \\
(485) \\
(486) \\
(487) \\
(488) \\
(489) \\
(490)
\end{array}$$

Fig. 2. Equality and Term Precision Rules (Selected)

the tedious reasoning associated with traditional step-indexing techniques. We provide a brief overview here; more details can be found in [6].

SGDT offers a synthetic approach to domain theory that allows for guarded recursion to be expressed syntactically via a type constructor $\triangleright : \text{Type} \rightarrow \text{Type}$ (pronounced “later”). The use of a modality to express guarded recursion was introduced by Nakano [24]. Given a type A , the type $\triangleright A$ represents an element of type A that is available one time step later. There is an operator $\text{next} : A \rightarrow \triangleright A$ that “delays” an element available now to make it available later. We will use a tilde to denote a term of type $\triangleright A$, e.g., $\tilde{M}$.

There is a *guarded fixpoint* operator

$$\text{fix} : \forall T, (\triangleright T \rightarrow T) \rightarrow T.$$

That is, to construct a term of type T , it suffices to assume that we have access to such a term “later” and use that to help us build a term “now”. This operator satisfies the axiom that $\text{fix } f = f(\text{next}(\text{fix } f))$. In particular, ~~this axiom applies to propositions~~ fix applies when type T is instantiated to a proposition $P : \text{Prop}$; ~~proving a statement in this manner is known as~~ in that case, it corresponds to Löb-induction.

3.1.1 Ticked Cubical Type Theory. Ticked Cubical Type Theory [21] is an extension of Cubical Type Theory [10] that has an additional sort called *ticks*. Ticks were originally introduced in [4]. A tick $t : \text{tick}$ serves as evidence that one unit of time has passed. In Ticked Cubical Type Theory, the type $\triangleright A$ is the type of dependent functions from ticks to A . The type A is allowed to depend on t , in which case we write $\triangleright_t A$ to emphasize the dependence.

The rules for tick abstraction and application are similar to those of ordinary Π types. A context now consists of ordinary variables $x : A$ as well as tick variables $t : \text{tick}$. The presence of the tick variable t in context $\Gamma, (t : \text{tick}), \Gamma'$ intuitively means that the values of the variables in Γ arrive “first”, then one time step occurs, and then the values of the variables in Γ' arrive. The abstraction

rule for ticks states that if in context $\Gamma, t : \text{tick}$ the term M has type A , then in context Γ the term $\lambda t.M$ has type $\triangleright_t A$. Conversely, if we have a term M of type $\triangleright A$, and we have available in the context a tick $t' : \text{tick}$, then we can apply the tick to M to get a term $M[t'] : A[t'/t]$. However, there is an important restriction on when we are allowed to apply ticks. ~~In order to:~~ To apply M to tick t , we require M ~~must-to~~ be well-typed in the prefix of the context occurring before the tick t . That is, all variables mentioned in M must be available before t . This ensures that we cannot, for example, define a term of type $\triangleright \triangleright A \rightarrow \triangleright A$ via repeated tick application. For the sake of brevity, we will also write tick application as M_t . The constructions we carry out in this paper will take place in a guarded type theory with ticks.

3.2 A Call-by-push-value Model

Our denotational model of GTLC must account for the fact that the language is call-by-value with non-trivial effects (divergence and errors). To do this, we formulate the model using the categorical structures of Levy's *call-by-push-value* [18]. Call-by-push-value is a categorical model of effects that is a refinement of Moggi's monadic semantics [23]. While Moggi works with a (strong) monad T on a category $\mathcal{V}$ of pure morphisms, call-by-push-value works instead with a decomposition of such a monad into an adjunction between a category $\mathcal{V}$ of "value types" and pure morphisms and a category $\mathcal{E}$ of "computation types" and linear morphisms. The monad T on $\mathcal{V}$ is decomposed into a left adjoint $F : \mathcal{V} \rightarrow \mathcal{E}$ constructing the type FA of computations that return A values and a right adjoint $U : \mathcal{E} \rightarrow \mathcal{V}$ constructing the type UB of values that are first-class suspended computations of type B . The monad T can then be reconstituted as $T = UF$ but the main advantage is that many constructions on effectful programs naturally decompose into constructions on value/computation types. Most strikingly, the CBV function type $A \rightarrow A'$ decomposes into the composition of three constructions $U(A \rightarrow FA')$ where $\rightarrow : \mathcal{V}^{op} \times \mathcal{E} \rightarrow \mathcal{E}$ is a kind of mixed kinded function type. This decomposition considerably simplifies treatment of the CBV function type. Additionally, prior work on gradual typing semantics has shown that the casts of gradual typing are naturally formulated in terms of pairs of a pure morphism and a linear morphism, as we explain below [30].

For our simple denotational semantics, the category $\mathcal{V}$ is simply the category **Set** of sets and functions. Then because GTLC is call-by-value, all types will be interpreted as objects of this category, i.e., sets. More interesting are the computation types. These will be interpreted in a category of sets equipped with ~~"algebraic"~~ algebraic³ structure with morphisms being functions that are homomorphic in this structure. This algebraic structure needs to model the effects present in our language, in this case these are errors as well as taking computational steps (potentially diverging by taking computational steps forever). We call these algebraic structures simple error domains to distinguish them from the error domains we define later to model graduality:

Definition 3.1 (Simple Error Domains). A (simple) error domain B consists of

- (1) A carrier set UB
- (2) An element $\mathcal{U}_B : UB$ representing error
- (3) A function $\theta_B : \triangleright UB \rightarrow UB$ modeling a computational step

A homomorphism of error domains $\phi : B \multimap B'$ is a function $\phi : UB \rightarrow UB'$ that preserves ~~these in that~~ $\mathcal{U}$ and θ , i.e., $\phi(\mathcal{U}_B) = \mathcal{U}_{B'}$ and $\phi(\theta_B(\tilde{x})) = \theta_{B'}(\triangleright(\phi(\tilde{x})))$. Simple error domains and homomorphisms assemble into a category **ErrDom** with a forgetful functor $U : \mathbf{ErrDom} \rightarrow \mathbf{Set}$ taking the underlying set and function.

Here is our first point where we utilize guarded type theory: rather than simply being a function $UB \rightarrow UB$, the "think" map θ takes an element *later*. This makes a major difference, because the

³~~the~~ The θ structure is not algebraic in the strictest sense since it does not have finite arity, ~~but,~~

structure of a think map combined with the guarded fixpoint operator allows us to define recursive elements of UB in that any function $f : UB \rightarrow UB$ has a “quasi-fixed point” $\text{qfix}(f) = \text{fix}(f \circ \theta_B)$ satisfying the quasi-fixed point property:

$$\text{qfix}(f) = f(\delta_B(\text{qfix}f))$$

where $\delta_B = \theta_B \circ \text{next}$ is a map we call the “delay” map which trivially delays an element now to be available later. As an example, we can define $\Omega_B = \text{qfix}(\text{id}) : UB$, the “diverging element” that “thinks forever” in that $\Omega_B = \delta(\Omega_B)\Omega_B = \delta_B(\Omega_B)$. We call this a “quasi” fixed point because it is *nearly* a fixed point except for the presence of the delay map δ_B , which is irrelevant from an extensional point of view where we would prefer to ignore differences in the number of steps that computations take.

Then to model effectful programs, we need to construct a left adjoint $L_U : \text{Set} \rightarrow \text{ErrDom}$ to U which constructs the “free error domain” on a set, which models an effectful CBV computation. We base the construction on the *guarded lift monad* described in [20]. Here, we augment the guarded lift monad to accommodate the additional effect of failure.

Definition 3.2 (Free error domain). For a set A , we define the (carrier of) *free error domain* $U(L_U A)$ as the unique solution to the guarded domain equation:

$$U(L_U A) \cong A + 1 + \triangleright U(L_U A).$$

For which we use the following notation for the three constructors:

- (1) $\eta : A \rightarrow U(L_U A)$
- (2) $\mathbb{U} : U(L_U A)$
- (3) $\theta : \triangleright U(L_U A) \rightarrow U(L_U A)$

Here $\mathbb{U}, \theta$ provide the error domain structure for $L_U A$, and this has the universal property of being free in that any function $f : U(L_U A) \rightarrow UB$ uniquely extends to a homomorphism $f^\dagger : L_U A \multimap B$ satisfying $Uf^\dagger \circ \eta = f$. In other words, the free error monad is left adjoint $L_U \dashv U$ to the forgetful functor, with η the unit of the adjunction. We will write $\theta_t(\dots)$ to mean $\theta(\lambda t. \dots)$.

An element of $L_U A$ should be viewed as a computation that can either (1) return a value (via η), (2) raise an error and stop (via $\mathbb{U}$), or (3) take a computational step (via θ). The operation $f^\dagger$ can be defined by guarded recursion in the apparent way, and proven to satisfy the freeness property by Löb induction.

The free error domain is essential to modeling CBV computation: a term $\Gamma \vdash M : A$ in GTLC will be modeled as a function $M : \llbracket \Gamma \rrbracket \rightarrow UL_U \llbracket A \rrbracket$ from the set denoted by Γ to the underlying set of the free error domain on the set denoted by A , just as in a monadic semantics with monad UL_U .

3.3 Modeling the type structure

Much of the type structure of GTLC is simple to model: Nat denotes the set of natural numbers and $\times$ denotes the cartesian product of sets. To model the CBV function type $A \multimap A'$ we use the CBPV decomposition $U(\llbracket A \rrbracket \rightarrow L_U \llbracket A' \rrbracket)$ where $A \rightarrow B$ is the obvious point-wise algebraic structure on the set of functions from A to UB .

The final type to model is the the dynamic type $?$. In the style of Scott, a classical domain theoretic model for $?$ would be given by solving a domain equation

$$D \cong \text{Nat} + (D \times D) + U(D \rightarrow L_U D),$$

representing the fact that dynamically typed values can be numbers, pairs or CBV functions. This equation does not have inductive or coinductive solutions due to the negative occurrence of D in the domain of the function type. In classical domain theory this is avoided by instead taking the

initial algebra in a category of embedding-projection pairs, by which we obtain an exact solution to this equation. However, in guarded domain theory we instead replace this domain equation with a guarded domain equation. Consider the following similar-looking equation:

$$D \cong \text{Nat} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}}D). \quad (1)$$

Since the negative occurrence of D is guarded under a later, we can solve this equation by a mixture of inductive types and guarded fixed points. Specifically, consider the parameterized inductive type

$$D' X := \mu T. \text{Nat} + (T \times T) + X.$$

Then we can construct D as the unique solution to the guarded domain equation

$$D \cong D'(\triangleright U(D \rightarrow L_{\mathcal{U}}D))$$

which is a guarded equation in that all occurrences of D on the right hand side are guarded by the $\triangleright$. Expanding out the guarded fixed point and least fixed point property gives us that D satisfies Equation 1. We then refer to the three injections for numbers, pairs and functions as $\text{Inj}_{\mathbb{N}}$, $\text{Inj}_{\times}$, $\text{Inj}_{\rightarrow}$ respectively.

3.4 Term Semantics

We now extend our semantics of types to a semantics of terms of GTLC. As mentioned above, terms $x : A_1, \dots \vdash M : A$ are modeled as effectful functions $\llbracket M \rrbracket : \llbracket A_1 \rrbracket \times \dots \rightarrow UL_{\mathcal{U}}\llbracket A \rrbracket$ and values $x : A_1, \dots \vdash V : A$ are also modeled as pure functions $\llbracket V \rrbracket : \llbracket A_1 \rrbracket \times \dots \rightarrow \llbracket A \rrbracket$. The interpretation of the typical CBV features of GTLC into a [model of CBPV](#) is standard [18]; the only additional part we need to account for is the semantics of casts, shown in Figure 3. Following the CBPV treatment of gradual typing given by New-Licata-Ahmed, an upcast $\text{up } c$ where $c : A \sqsubseteq A'$ is interpreted as a pure function $\llbracket \text{up } c \rrbracket : \llbracket A \rrbracket \rightarrow \llbracket A' \rrbracket$ whereas a downcast $\text{dn } c$ is interpreted as a homomorphism $\llbracket \text{dn } c \rrbracket : L_{\mathcal{U}}\llbracket A' \rrbracket \multimap L_{\mathcal{U}}\llbracket A \rrbracket$. We define the semantics of upcasts and downcasts simultaneously by recursion over type precision derivations. The reflexivity casts are given by identities, and the transitivity casts by composition. The upcasts for products, and the downcasts for functions are given by the functorial actions of the type constructors on the casts for the subformulae. The upcasts for functions and similarly the downcasts for products, on the other hand, are more complex. For these, we use not the ordinary functorial action of type constructors, but an action we call the *Kleisli* action.

The Kleisli action of type constructors can be defined in any CBPV model. First, we can define the Kleisli category of value types $\mathcal{V}_k$ to have value types as objects, but have morphisms $\mathcal{V}_k(A, A') = \mathcal{E}(FA, FA')$. Similarly, we can define a Kleisli category of computation types $\mathcal{E}_k$ to have computation types as objects, but have morphisms $\mathcal{E}_k(B, B') = \mathcal{V}(UB, UB')$. The intuition is that these are the “effectful” morphisms between value/computation types, whereas the morphisms of $\mathcal{V}/\mathcal{E}$ are “pure”/“linear”. Then every CBPV type constructor $F, U, \times, \rightarrow$ has in addition to its usual functorial action on pure/linear morphisms, an action in each argument on Kleisli morphisms. So for instance, the function construction is functorial in pure/linear morphisms $\rightarrow : \mathcal{V}^{\text{op}} \times \mathcal{E} \rightarrow \mathcal{E}$ but on Kleisli morphisms what we have is for each computation type B a functorial action $- \rightarrow B : \mathcal{V}_k^{\text{op}} \rightarrow \mathcal{E}_k$ given by $- \xrightarrow{k} B$ on objects and for each value type a functorial action $A \xrightarrow{k} - : \mathcal{E}_k \rightarrow \mathcal{E}_k$. Similarly we get two actions $A \xrightarrow{k} -$ and $- \xrightarrow{k} A'$ on Kleisli morphisms between value types. In these two cases we do not get a bifunctorial action because the two Kleisli actions do not commute past each other, i.e., in general $(A'_l \xrightarrow{k} f_r \circ f_l \xrightarrow{k} A_r) \neq (f_l \xrightarrow{k} A'_r \circ A_l \xrightarrow{k} f_r)$. The full definitions of the Kleisli actions are included in Appendix B.

$$\begin{aligned}
\llbracket \text{up } r(A) \rrbracket &= \text{id}_{\underline{A} \llbracket A \rrbracket} \\
\llbracket \text{up } (cc') \rrbracket &= \llbracket \text{up } c' \rrbracket \circ \llbracket \text{up } c \rrbracket \\
\llbracket \text{up } (c_i \rightarrow c_o) \rrbracket &= (\llbracket \text{dn } c_i \rrbracket \xrightarrow{k} L_U \llbracket A'_o \rrbracket) \circ (\llbracket A_i \rrbracket \xrightarrow{k} UL_U(\llbracket \text{up } c_o \rrbracket)) \\
\llbracket \text{up } (c_1 \times c_2) \rrbracket &= \llbracket \text{up } c_1 \rrbracket \times \llbracket \text{up } c_2 \rrbracket \\
\llbracket \text{up } \text{Inj}_{\mathbb{N}} \rrbracket &= \text{Inj}_{\mathbb{N}} \\
\llbracket \text{up } \text{Inj}_{\times} \rrbracket &= \text{Inj}_{\times} \\
\llbracket \text{up } \text{Inj}_{\rightarrow} \rrbracket &= \text{Inj}_{\rightarrow} \circ \text{next} \\
\llbracket \text{dn } r(A) \rrbracket &= \text{id}_{\underline{F} \underline{A} L_U \llbracket A \rrbracket} \\
\llbracket \text{dn } (cc') \rrbracket &= \llbracket \text{dn } c \rrbracket \circ \llbracket \text{dn } c' \rrbracket \\
\llbracket \text{dn } (c_i \rightarrow c_o) \rrbracket &= U(\llbracket \text{up } c_i \rrbracket \rightarrow \llbracket \text{dn } c_o \rrbracket) \\
\llbracket \text{dn } (c_1 \times c_2) \rrbracket &= (\text{dn } c_1 \xrightarrow{k} A_2) \circ (A'_1 \xrightarrow{k} \text{dn } c_2) \\
\llbracket \text{dn } \text{Inj}_{\mathbb{N}} \rrbracket &= (\lambda V_d. \text{case } (V_d) \text{ of } \{\text{Inj}_{\mathbb{N}} n \rightarrow \eta n \mid \text{otherwise} \rightarrow \mathcal{U}\})^\dagger \\
\llbracket \text{dn } \text{Inj}_{\times} \rrbracket &= (\lambda V_d. \text{case } (V_d) \text{ of } \{\text{Inj}_{\times} (d_1, d_2) \rightarrow \eta (d_1, d_2) \mid \text{otherwise} \rightarrow \mathcal{U}\})^\dagger \\
\llbracket \text{dn } \text{Inj}_{\rightarrow} \rrbracket &= (\lambda V_d. \text{case } (V_d) \text{ of } \{\text{Inj}_{\rightarrow} \tilde{f} \rightarrow \theta_t (\eta(\tilde{f}_t)) \mid \text{otherwise} \rightarrow \mathcal{U}\})^\dagger
\end{aligned}$$

Fig. 3. Semantics of casts

Lastly, we have the upcasts and downcasts for the injections into the dynamic type, which are the core primitive casts. The upcasts are simply the injections themselves, except the function case which must include a next to account for the fact that the functions are under a later in the dynamic type. The downcasts are similar in that on values they pattern match on the input and return it if it is of the correct type, otherwise erroring. Again, the function case is slightly different in that if the input is in the function case, then it is actually only a function available later, and so we must insert a “think” in order to return it.

3.5 Extracting a well-behaved big-step semantics

We now define from the above term model a “big-step” semantics. More concretely, our goal is to define a [partial](#) function $- \Downarrow : \{M \mid \cdot \vdash M : \text{Nat}\} \rightarrow \mathbb{N} + \mathcal{U}$.

To begin, we must first discuss another aspect of guarded type theory, *clocks*, originally introduced in [3]. The constructions of guarded type theory ($\triangleright$, next, and fix) we have been using may be indexed by objects called *clocks*. A clock intuitively serves as a reference relative to which steps are counted. For instance, given a clock k and type X , the type $\triangleright^k X$ represents a value of type X one unit of time in the future according to clock k . Up to this point, all uses of guarded constructs have been indexed by a single clock k . In particular, we have for each clock k a type $L_U^k X$. The notion of *clock quantification* allows us to pass from the guarded setting to the usual set-theoretic world. Clock quantification enables us to encode coinductive types using guarded recursion. That is, given a type A with a free variable $k : \text{Clock}$, we can form the type $A^{gl} := \forall k. A$ ([here, gl stands for “global”](#)). Given a functor F that satisfies a technical condition, if A is a guarded recursive type satisfying $A \cong F(\triangleright^k A)$, then the type A^{gl} has a final F -coalgebra structure (see Theorem 4.3 of [5]).

Consider the functor $F(X) = \mathbb{N} + \mathcal{U} + X$. By the definition of the guarded lift, we have that $L_U^k \mathbb{N} \cong \mathbb{N} + \mathcal{U} + \triangleright^k (L_U \mathbb{N}) \cong F(\triangleright L_U^k \mathbb{N})$. Furthermore, it can be shown that the functor F satisfies the technical condition in the statement of the above-mentioned theorem. Thus, the theorem

implies that the type $L_{\mathcal{U}}^{gl}\mathbb{N} := \forall k. L_{\mathcal{U}}^k\mathbb{N}$ is a final F -coalgebra. In fact, $L_{\mathcal{U}}^{gl}$ is isomorphic to a more familiar coninductive type. Given a type Y , Capretta's *delay monad* [8], which we write as $\text{Delay}(Y)$, is defined as the coinductive type generated by $\text{now} : Y \rightarrow \text{Delay}(Y)$ and $\text{later} : \text{Delay}(Y) \rightarrow \text{Delay}(X)$. We have that $\text{Delay}(\mathbb{N} + \mathcal{U})$ is a final coalgebra for the functor F defined above. This means $L_{\mathcal{U}}^{gl}\mathbb{N}$ is isomorphic to $\text{Delay}(\mathbb{N} + \mathcal{U})$. Denote this isomorphism by α .

We can Given $d : \text{Delay}(\mathbb{N} + \mathcal{U})$, we define a notion of termination in i steps for the Delay monad $d \Downarrow^i n_?$ where $d : \text{Delay}(\mathbb{N} + \mathcal{U})$ and, written $d \Downarrow^i$. We define $d \Downarrow^i$ inductively by the rules (now $n_?$) $\Downarrow^0$ and (later d) $\Downarrow^{i+1}$ if $d \Downarrow^i$ (here, $n_? : \mathbb{N} + \mathcal{U}$). From this, can definition, it is clear that if there is an i such that $d \Downarrow^i$, then this i is unique. Using this definition of termination, we define a partial function $\text{Delay}(\mathbb{N} + \mathcal{U}) \rightarrow \mathbb{N} + \mathcal{U}$ $f : \text{Delay}(\mathbb{N} + \mathcal{U}) \rightarrow \mathbb{N} + \mathcal{U}$ by existentially quantifying over i for which $d \Downarrow^i n_?$. That is, we define $f(d) := n_?$ if $d \Downarrow^i n_?$ for some (necessarily unique) i , and f is undefined otherwise.

Now consider the function $\{M \mid \vdash M : \text{Nat}\} \rightarrow L_{\mathcal{U}}^{gl}\mathbb{N}$ defined by $\lambda M. \lambda k. \llbracket M \rrbracket$ global version of the term semantics, $\llbracket \cdot \rrbracket^{gl} : \{M \mid \vdash M : \text{Nat}\} \rightarrow L_{\mathcal{U}}^{gl}\mathbb{N}$, defined by $\llbracket M \rrbracket^{gl} := \lambda(k : \text{Clock}). \llbracket M \rrbracket$. Composing this with the above partial function isomorphism α and the partial function f yields the desired partial function $\Downarrow : \{M \mid \vdash M : \text{Nat}\} \rightarrow \mathbb{N} + \mathcal{U}$. It is then straightforward to see that this big-step semantics respects the equational theory and that the denotations of the diverging term Ω , the error term $\mathcal{U}$, and a syntactic natural number value n are all distinct.

4 CHALLENGES TO A GUARDED MODEL OF GRADUALITY

Now that we have seen how to model the terms without regards to graduality, we next turn to how to enhance this model to provide a compositional semantics that additionally satisfies graduality. By a compositional semantics, we mean that we want to provide a compositional interpretation of *type* and *term precision* such that we can extract the graduality relation from our model of term precision.

4.1 A first attempt: Modeling Term Precision with Posets

In prior work, New and Licata model the term precision relation by equipping ω -CPOs with a second ordering $\sqsubseteq$ they call the “error ordering” such that a term precision relationship at a fixed type $M \sqsubseteq M' : r(A)$ implies the error ordering $\llbracket M \rrbracket \sqsubseteq \llbracket M' \rrbracket$ in the semantics. Then the error ordering on the free error domain $L_{\mathcal{U}}A$ is based on the graduality property: essentially either $\llbracket M \rrbracket$ errors or both terms diverge or both terms terminate with values related by the ordering relation on A . The obvious first try at a guarded semantics that models graduality would be to similarly enhance our value types and computation types to additionally carry a poset structure. For most type formers this ordering can be defined as lifting poset structures on the subformulae; the key design choice is in the ordering on the free error domain $L_{\mathcal{U}}A$.

The ordering on $L_{\mathcal{U}}A$ should model the graduality property, but how to interpret this is not so straightforward. The graduality property as stated mentions divergence, but we only work indirectly with diverging computations in the form of the think maps. The closest direct encoding of the graduality property is an ordering we call the *step-insensitive error ordering*, defined by guarded recursion in Figure 4. Firstly, to model graduality, $\mathcal{U}$ is the least element. Next, two returned values are related just when they are in the ordering $\sqsubseteq$ on A . Two thinking elements are related just when they are related later. The interesting cases are then those where one side is thinking and the other has completed to either an error or a value. Since the graduality property is *extensional*, i.e., oblivious to the number of steps taken, it is sensible to say that if one side has terminated and the other side is thinking, that we must require the thinking side to eventually terminate with a related behavior, which is the content of the final 4 three cases of the definition.

$$\begin{aligned}
& \mathbb{U} \lesssim l \text{ iff } \top \\
& \eta x \lesssim \eta y \text{ iff } x \sqsubseteq y \\
& \theta \tilde{l} \lesssim \theta \tilde{l}' \text{ iff } \triangleright_t(\tilde{l}_t \lesssim \tilde{l}'_t) \\
& \theta \tilde{l} \lesssim \mathbb{U} \text{ iff } \exists n. \theta \tilde{l} = \delta^n(\mathbb{U}) \\
& \theta \tilde{l} \lesssim \eta y \text{ iff } \exists n. \exists x \sqsubseteq y. (\theta \tilde{l} = \delta^n(\eta x)) \\
& \eta x \lesssim \theta \tilde{l}' \text{ iff } \exists n. \exists y \sqsupseteq x. (\theta \tilde{l}' = \delta^n(\eta y))
\end{aligned}$$

Fig. 4. Step-insensitive error ordering

This definition of $\lesssim$ is sufficient to model the graduality property in that if $l \lesssim l'$ for $l, l' : L_{\mathbb{U}}\mathbb{N}$ where $\mathbb{N}$ is the flat poset whose ordering is equality, then the big-step semantics we derive for l, l' does in fact satisfy the intended graduality relation. However, there is a major issue with this definition: it's not a poset at all: it is not anti-symmetric, but more importantly it is *not* transitive! To see why, we observe the following undesirable property of any relation that is transitive and “step-insensitive” on one side:

THEOREM 4.1. *Let R be a binary relation on the free error domain $U(L_{\mathbb{U}}A)$. If R satisfies the following properties*

- (1) *Transitivity*
- (2) *θ -congruence: If $\triangleright_t(\tilde{x}_t R \tilde{y}_t)$, then $\theta(\tilde{x}) R \theta(\tilde{y})$.*
- (3) *Right step-insensitivity: If $x R y$ then $x R \delta y$.*

Then for any $l : U(L_{\mathbb{U}}A)$, we have $l R \Omega$. If R is left step-insensitive instead then $\Omega R x$.

PROOF. By Löb induction, ~~in the appendix:~~ we assume that l is related to Ω later, which implies that $\theta(\text{next } l) R \theta(\text{next } \Omega)$. Then observe that $l R \theta(\text{next } l) R \theta(\text{next } \Omega) = \Omega$. See the appendix for more details. $\square$

COROLLARY 4.2. *Let R be a binary relation on $U(L_{\mathbb{U}}A)$. If R satisfies transitivity, θ -congruence and left and right step-insensitivity, then R is the total relation: $\forall x, y. x R y$.*

PROOF. $x R \Omega$ and $\Omega R y$ by the previous theorem. Then by transitivity $x R y$. $\square$

This in turn implies $\lesssim$ is not transitive, since it is a θ -congruence and step-insensitive on both sides, but not trivial: e.g., for the flat poset on $\mathbb{N}$, $\eta 0 \lesssim \eta 1$ is false. This shows definitively that we cannot provide a compositional semantic graduality proof based on types as posets if we use this step-insensitive ordering on $L_{\mathbb{U}}A$.

4.2 Why transitivity is essential

At this point we might try to weaken ~~from~~ our initial attempt at providing a poset semantics to a semantics where types are equipped with merely a reflexive relation, and continue using the step-insensitive ordering. However, some level of transitivity is absolutely essential to providing compositional reasoning, and transitivity is used pervasively in the prior work by New and Licata. The reason that it is essential ~~has to do with~~ becomes clear by examining the details of the graduality proof, so first we expand on how we prove graduality compositionally. First ~~of all~~, we need to extend our compositional semantics of types and terms to ~~also~~ a compositional semantics of *type precision derivations* and *term precision*. Sticking to a poset-based semantics, if our value types

are ~~posets, we then interpreted as posets, then we~~ want our precision derivations $c : A \sqsubseteq A'$ to be ~~interpreted as a poset relation on A, A' , that is relations between the underlying sets of A, A' that are relation between A and A'~~ . It is also natural to ask that the relation c interact with the orderings on A and A' . More specifically, if $x' \sqsubseteq_A x$ and $x c y$, then $x' c y$. Similarly, if $x c y$ and $y \sqsubseteq_{A'} y'$, then $x c y'$. We summarize this requirement by saying c is downward-closed in A and upward-closed upward-closed in A' . We ~~denote~~ will see below why this requirement is necessary for the graduality proof.

We call such a relation between A and A' by a *poset relation*, and we write $c : A \multimap A'$. The paradigmatic example of a poset relation is ~~the poset ordering itself, $\sqsubseteq_A : A \multimap A$, which given by the ordering $\sqsubseteq_A$~~ . We call this the *reflexive poset relation* and denote it by $r(A) : A \multimap A$. This is a poset relation because ~~it the ordering $\sqsubseteq_A$~~ is transitive.

Given two poset relations $c : A_1 \multimap A_2$ and $c' : A_2 \multimap A_3$, we can form their *composition* in the usual manner, i.e., $x c c' y$ if and only if there is an element $z : A_2$ with $x c z$ and $z c' y$.

These relations are then used to model term precision. For closed programs $M \sqsubseteq M' : c$ where $c : A \sqsubseteq A'$, the terms denote elements of the free error domain $\llbracket M \rrbracket, \llbracket M' \rrbracket : UL_U \llbracket A \rrbracket$, and we model the term precision semantically as the relation holding between those denoted elements $\llbracket M \rrbracket (UL_U \llbracket c \rrbracket) \llbracket M' \rrbracket$ ~~where. Here $UL_U \llbracket c \rrbracket$ is a relational lifting of $U \circ L_U$, which extends $\llbracket c \rrbracket$ to account for error and computational steps~~. More generally, M and M' can be *open* terms, in which case they denote functions, not just elements. We model that as not simply elements being related but in a typical logical-relations style as *preserving* relatedness: if they are passed in inputs related by the domain relations then their output is related by the codomain relation. To capture this preservation of relations property, we use the notion of a *square*, which specifies a relation between two monotone functions. The definition of square is as follows. Let A_i, A_o, A'_i, A'_o be partially-ordered sets. Let $c_i : A_i \multimap A'_i$ and $c_o : A_o \multimap A'_o$ be poset relations, and let $f : A_i \rightarrow A_o$ and $g : A'_i \rightarrow A'_o$ be monotone functions. We say that $f \sqsubseteq_{c_o c_i}^{c_i} g$ if for all $x : A_i$ and $y : A'_i$ with $x c_i y$, we have $f(x) c_o g(y)$. We call this a *square* because we visualize this situation as follows:

$$\begin{array}{ccc} A_i & \xrightarrow{c_i} & A'_i \\ f \downarrow & & \downarrow g \\ A_o & \xrightarrow{c_o} & A'_o \end{array}$$

Squares are then used to model the term precision ordering in that $\Delta \vdash M \sqsubseteq N : c$ implies $\llbracket M \rrbracket \sqsubseteq_{UL_U \llbracket c \rrbracket}^{\llbracket \Delta \rrbracket} \llbracket N \rrbracket$ where here $\llbracket \Delta = x_1 : c_1, \dots \rrbracket$ is the action of the product on the relations $c_1, \dots$.

We note that for any c , there is a “vertical identity” square $\text{id} \sqsubseteq_c^c \text{id}$, as can be seen by unfolding the definition. Similarly, for any ~~montone-monotone~~ function $f : A_i \rightarrow A_o$, there is a “horizontal identity” square $f \sqsubseteq_{\leq_{A_o}}^{\leq_{A_i}} f \sqsubseteq_{r(A_o)}^{r(A_i)} f$ arising from the fact that f is monotone. We can *compose* squares vertically, i.e., if we have $f \sqsubseteq_{c_2}^{c_1} f'$ and $g \sqsubseteq_{c_3}^{c_2} g'$ then we obtain a square $g \circ f \sqsubseteq_{c_3}^{c_1} g' \circ f'$. Likewise, we can compose squares horizontally: If $f \sqsubseteq_{c_o}^{c_i} g$ and $g \sqsubseteq_{c_o'}^{c_i'} h$, then we obtain a square $f \sqsubseteq_{c_o c_o'}^{c_i c_i'} h$. Horizontal composition of squares corresponds to transitivity of term precision. While we do not dwell on categorical abstractions in this work, we note that this structure of functions, relations and squares forms a locally thin *double category* as used in previous work [29].

Now that we have an intended model of term precision in the form of squares, we can see what is required to give a compositional proof of graduality. Most of the cases of term precision are just congruence and follow easily. The core of the proof of the graduality property is in proving the validity of the *cast rules* **UpL**, **UpR**, **DnL** and **DnR** **UpL**, **UpR**, **DnL**, and **DnR**. These rules specify a relationship between the semantics of type precision derivations c and the corresponding casts

up c , dn c (see Section 2). To prove graduality compositionally then, we will need to codify in an extra property on $\llbracket c \rrbracket$ that the relation $\llbracket c \rrbracket$ has some correspondence with the casts $\llbracket \text{up } c \rrbracket$, $\llbracket \text{dn } c \rrbracket$. What property should we require?

Consider the UpL-UpL rule. It states that if $c : A_1 \sqsubseteq A_2$ and $c' : A_2 \sqsubseteq A_3$ and $M : A_1$ is related to $N : A_3$ via the composition cc' , then the upcast $\text{up } c \ M$ is related to N via c' . Since the upcasts are pure functions in the semantics this can be deduced from the existence of the following square, where the relation on the top is $\llbracket c \rrbracket \llbracket c' \rrbracket$:

$$\begin{array}{ccc} \llbracket A_1 \rrbracket & \xrightarrow{\llbracket c \rrbracket} & \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & \llbracket A_3 \rrbracket \\ \llbracket \text{up } c \rrbracket \downarrow & & & & \downarrow \text{id} \\ \llbracket A_2 \rrbracket & \xrightarrow{\llbracket c' \rrbracket} & \llbracket A_3 \rrbracket & & \end{array}$$

While at first look this seems to be specifying a relationship between $\llbracket c \rrbracket$ and $\llbracket \text{up } c \rrbracket$, there is a problem: it also quantifies involves-over an arbitrary other relation $\llbracket c' \rrbracket$! This means we cannot require the existence of this square as part of the definition of a relation between value types, as it is self-referential. This would seem to imply that we cannot give a compositional model for graduality. However, New and Licata observed that in the presence of transitivity, we can *derive* the above squares from simpler ones that do not involve composition of relations. Below is the square corresponding to UpL:

$$\begin{array}{ccc} A_1 & \xrightarrow{c} & A_2 \\ \text{up } c \downarrow & & \downarrow \text{id} \\ A_2 & \xrightarrow{\leq_{A_2}} & A_2 \end{array}$$

are the simpler squares corresponding to UpL, UpR, DnL, and DnR:

$$\begin{array}{c} \begin{array}{ccc} A_1 & \xrightarrow{c} & A_2 \\ \text{up } c \downarrow & \text{UpL} & \downarrow \text{id} \\ A_2 & \xrightarrow{r(A_2)} & A_2 \end{array} \quad \left| \quad \begin{array}{ccc} A_1 & \xrightarrow{r(A_1)} & A_1 \\ \text{id} \downarrow & \text{UpR} & \downarrow \text{up } c \\ A_1 & \xrightarrow{c} & A_2 \end{array} \quad \left| \quad \begin{array}{ccc} A_2 & \xrightarrow{r(A_2)} & A_2 \\ \text{dn } c \downarrow & \text{DnL} & \downarrow \text{id} \\ A_1 & \xrightarrow{c} & A_2 \end{array} \quad \left| \quad \begin{array}{ccc} A_2 & \xrightarrow{c'} & A_3 \\ \text{id} \downarrow & & \downarrow \text{id} \\ A_2 & \xrightarrow{c'} & A_3 \end{array} \right. \end{array}$$

These squares say that the relation c is *representable* by the upcast morphism $\text{up } c$, essentially that the relation is a kind of *graph* of the function in that $x \llbracket c \rrbracket y$ if and only if $\llbracket \text{up } c \rrbracket(x) \leq_{A_2} y$ [31].

We can use this simpler the simpler UpL square to derive the original UpL square as long as we can horizontally compose UpL square by horizontally composing with the identity square for c' :

$$\begin{array}{ccccc} A_1 & \xrightarrow{c} & A_2 & \xrightarrow{c'} & A_3 \\ \text{up } c \downarrow & & \text{id} \downarrow & & \downarrow \text{id} \\ A_2 & \xrightarrow{\sqsubseteq_{A_2}} & A_2 & \xrightarrow{c'} & A_3 \end{array}$$

Then to complete the proof, we observe that the composition of $\sqsubseteq_{A_2}$ with c' is equal to c' , because c' is *downward-closed* under the relation on A_2 . An analogous argument shows that we can derive the original UpR square from the simpler version of UpR, this time using the fact that poset relations are upward-closed.

The actual place where transitivity of the relations on the value types is needed in this argument occurs when $A_3 = A_2$ and we take c' to be $\sqsubseteq_{A_2}$ (recall that c' is universally quantified, so it can be any relation). The fact that $\sqsubseteq_{A_2}$ is downward-closed is equivalent to it being transitive. In other

	$\boxed{l_1 \approx l_2}$	
	$\mathcal{U} \approx \mathcal{V} \text{ iff } \top$	
	$\eta x \approx \eta y \text{ iff } x \approx_A y$	
	$\theta \tilde{x} \approx \theta \tilde{y} \text{ iff } \triangleright_t(\tilde{x}_t \approx \tilde{y}_t)$	
	$\theta \tilde{x} \approx \mathcal{U} \text{ iff } \exists n. \theta \tilde{x} = \delta^n(\mathcal{U})$	
	$\theta \tilde{x} \approx \eta y \text{ iff } \exists n. \exists x \approx_A y. (\theta \tilde{x} = \delta^n(\eta x))$	
	$\mathcal{U} \approx \theta \tilde{y} \text{ iff } \exists n. \theta \tilde{y} = \delta^n(\mathcal{U})$	
	$\eta x \approx \theta \tilde{y} \text{ iff } \exists n. \exists y \approx_A x. (\theta \tilde{y} = \delta^n(\eta y))$	

Fig. 5. lock-step error ordering and weak bisimilarity

words, the relations $\sqsubseteq_A$ on value types must be poset relations. To recap, we have shown that in order to carry out the above compositional construction.

So we see that in order to get a sensible condition that can give us a compositional definition of a well-behaved relation, some amount of transitive reasoning on squares is required, and graduality proof compositionally, we need that the relations are closed under the ordering on each side. In particular, this implies that the ordering relations $\sqsubseteq_A$ must be transitive, so we cannot entirely drop posets from our semantics.

4.3 Resolution: Splitting Error Ordering and Bisimilarity

Given that some amount of transitive reasoning is essential for modeling type and term precision compositionally, we need to revisit our ordering relation on $L_{\mathcal{U}}A$ to get one that is transitive. Theorem 4.1 tells us, however, that any non-trivial such relation must either not be a congruence with respect to θ , or must not be left- and right-step-insensitive.⁴ We cannot forego the property of being a θ -congruence, as without this we would not be able to prove basic properties of the relation using Löb-induction, e.g., that the extension $f^\dagger$ is *monotone*, which is used pervasively in the proof of graduality. Thus, we choose to sacrifice left and right step-insensitivity. That is, we will define an ordering relation that requires terms to be in “lock-step”. In order for two computations to be related in this ordering, they must have the exact same stepping behavior (unless/until the left-hand side results in an error).

More formally, we define the *lock-step error-ordering*, with the idea being that two computations l and l' are related if they are in lock-step with regard to their intensional behavior, up to l erroring. Figure 5 gives the definition of this relation.

When both sides are η , then we check that the returned values are related in $\sqsubseteq_A$. The error term $\mathcal{U}$ is the least element. Lastly, if both sides step (i.e., are a θ) then we compare the resulting computations one time step later. It is straightforward to prove using Löb-induction that this relation is reflexive, transitive and anti-symmetric given that the underlying relation R underlying relation $\sqsubseteq_A$ has those properties. The lock-step ordering is therefore the partial ordering we will associate with $L_{\mathcal{U}}A$. More generally we can define a heterogeneous version of this ordering that lifts poset

⁴Technically, the theorem only implies that we need to drop one of left- or right-step-insensitivity. We might then attempt to work with two separate “step-semi-sensitive” relations on $L_{\mathcal{U}}A$, each closed under delays on the left and right respectively. This has some similarities to prior work on logical relations models [27, 30], but we do not take this approach because the relations still do not seem to be provably transitive, though we have not been able to prove that they are not transitive either.

relation $c : A \multimap A'$ to a poset relation $L_U c : L_U A \multimap L_U A'$. The definition is nearly identical, except that in the case when both sides are η we check that the returned values are related in c .

However, we also cannot completely drop *step-insensitivity* from our model. The graduality property itself is not sensitive to steps, and in fact there are terms related by term precision that take differing numbers of steps. The offending step arises precisely from the place where our definition of D uses a $\triangleright$: the function case. Consider the representability condition corresponding to the **DnL** rule for $\text{Inj}_{\rightarrow} : ? \rightarrow ? \sqsubseteq ?$. This is the square $\llbracket \text{dn Inj}_{\rightarrow} \rrbracket \sqsubseteq_{L_U \llbracket \text{Inj}_{\rightarrow} \rrbracket}^{r(L_U D)} \text{id}$. For this square to be valid with the lock-step ordering the left and right hand side would need to take the same number of steps, at least when the left-hand side does not error. However, the right hand side *always* takes 0 steps, as it is the identity function, whereas on the left, our definition of $\llbracket \text{dn Inj}_{\rightarrow} \rrbracket$ had to take an observable step when its input is a function: this was inherent to the fact that the function case of the dynamic type is guarded by a later.

However, observe that we can remedy this particular situation by replacing the id on the right hand side by an innocuous function $\delta^* = (\delta \circ \eta)^\dagger$ that on an input value takes a single computational step, but is otherwise the identity function. If we were to ignore computational steps, then this function *would* be the identity function. We call such a function that is the identity function except for the introduction of computational steps a *perturbation*.

We formalize this property of being equivalent “except for steps” with a second relation on the free error domain $L_U A$: *weak bisimilarity*, defined in Figure 5, which is parameterized by a binary relation $\approx_A$ on A [20]. Two errors are bisimilar, and when both sides are η , we ensure that the underlying values are bisimilar in the underlying bisimilarity relation on A . When both sides are thinking, we ensure the terms are bisimilar later. Most importantly, when one side is thinking but the other terminates at ηx (i.e., one side steps), we stipulate that the θ -term runs to ηy where x is related to y . And similarly, if one side is thinking and the other errors, we ensure the thinking side eventually errors.

It can be shown (by Löb-induction) that weak bisimilarity is reflexive and symmetric. Since it is non-trivial, a θ congruence and step-insensitive on both sides, by Theorem 4.1, we also know that it is *not* transitive. We will then require our denotations of types to be not just posets, but posets additionally equipped with a reflexive symmetric relation $\approx_A$.

Then we can refine our denotation of term precision $\Delta \vdash M \sqsubseteq N : c$ to mean not that $\llbracket M \rrbracket$ and $\llbracket N \rrbracket$ are necessarily in a lock-step error ordering directly, but that they can be “synchronized” to do so, that is that there exist $f \approx \llbracket M \rrbracket$ and $g \approx \llbracket N \rrbracket$ such that $f \sqsubseteq_{UL_U \llbracket c \rrbracket}^{\llbracket \Delta \rrbracket} g$. Note that this in turn implies that $\llbracket M \rrbracket$ and $\llbracket N \rrbracket$ are related in the original *step-insensitive* error ordering that we sought to prove! We additionally weaken our representability squares so that our upcasts and downcasts are not required to be in a lock-step ordering with an identity but instead to be in a lock-step ordering with some perturbation, that is a function *bisimilar* to the identity. We call this weakened form of representability *quasi-representability*. Then if our relations are quasi-representable we can in fact prove the validity of our cast rules. The remainder of the proof of graduality then is to show that this property of being quasi-representable is itself compositional: that all of the constructions we have on type precision preserve the property of being quasi-representable. While this is true for representability, it is not quite true for quasi-representability. The reason is that perturbations are not quite as well behaved as *actual* identity functions. To solve this final issue, we attach one final piece of information to our types A : a type of *syntactic* perturbations that represent functions bisimilar to the identity, but are presented as data so that we can perform operations such as composition and functorial actions on them. With this notion of syntactic perturbation, and additional requirements that the relations interact well with perturbations, we finally achieve our desired result: a compositional, syntax-independent proof of graduality.

5 A GUARDED MODEL OF GRADUALITY

Next we describe in detail our compositional model of GTLC that satisfies graduality. The definitions proceed in two phases: in the first phase we define our objects, functions, relations and squares to model the error ordering and bisimilarity, and then we add the extra structure of perturbations and quasi-representability to our objects and relations to ensure that we can model the cast rules. Once these definitions are complete we show how to interpret the types, type precision, terms and term precision compositionally in our model, and finally we conclude by extracting a big step semantics from this model that satisfies the graduality theorem.

5.1 Phase One: Predomains and Error Domains

As discussed in the previous section our model of types must now come with both an error ordering and a bisimilarity relation.

Definition 5.1. A **predomain** A consists of a set A along with two relations:

- A partial order $\sqsubseteq_A$.
- A reflexive, symmetric “bisimilarity” relation $\approx_A$.

A **morphism of predomains** $f : A \rightarrow A'$ is a function between the underlying sets that is both *monotone* (if $x \sqsubseteq_A x'$, then $f(x) \sqsubseteq_{A'} f(x')$), and *extensional* (if $x \approx_A x'$, then $f(x) \approx_{A'} f(x')$).

We will generally write A for both a predomain and its underlying set; if we need to emphasize the difference we will write $|A|$ for the underlying set of A . Given a predomain A , we can form the predomain $\triangleright A$. The underlying set is $\triangleright |A|$ and the ordering is given by $\tilde{x} \sqsubseteq_{\triangleright A} \tilde{x}'$ iff $\triangleright_t(\tilde{x}_t \sqsubseteq_A \tilde{x}'_t)$. Likewise, and likewise for bisimilarity.

We similarly enhance our error domains with ordering and bisimilarity relations, but we further need to impose conditions on the algebraic structure: the error element should be the least element (so that it models graduality), the think map should be monotone and extensional and the bisimilarity relation should relate δ to the identity (so that it models bisimilarity).

Definition 5.2. An **error domain** B consists of a predomain B along with the following data:

- A distinguished “error” element $\mathsf{U}_B \in B$ such that $\mathsf{U} \sqsubseteq_B x$ for all x
- A morphism of predomains $\theta_B : \triangleright B \rightarrow B$
- For all $x : B$ we have $\theta_B(\text{next } x) \approx_B x$

A **morphism of error domains** is a morphism of the underlying predomains that preserves the error element and the θ map.

For an error domain B , we define the predomain morphism $\delta_B := \theta_B \circ \text{next}$.

A *predomain relation* $c : A \multimap A'$ is a ~~monotone relation~~ relation between A and A' that is upward closed in A' and downward closed in A , as defined in the previous section (no condition is required for bisimilarity). The “reflexive” predomain relation is written $r(A)$ and is given by the ordering $\sqsubseteq_A$. The relational composition of two ~~predomains is also monotone~~ predomain relations is also a predomain relation, and $r(A)$ is the identity for this composition.

Error domain relations are given by predomain relations that are additionally a congruence with respect to the algebraic structures θ and U .

Definition 5.3. An **error domain relation** $d : B \multimap B'$ is a monotone relation d on the underlying predomains satisfying

- (1) (U congruence): $\mathsf{U}_B d \mathsf{U}_{B'}$.
- (2) (θ congruence): For all $\tilde{x} \in \triangleright B$ and $\tilde{y} \in \triangleright B'$, if $\triangleright_t(\tilde{x}_t d \tilde{y}_t)$, then $(\theta_B(\tilde{x}) d \theta_{B'}(\tilde{y}))$.

$$\begin{array}{c}
\frac{x \ d \ y \quad y \ d' \ z}{x \ (d \odot d') \ z} \quad \frac{x' \sqsubseteq_{B_1} x \quad x \ (d \odot d') \ z}{x' \ (d \odot d') \ z} \quad \frac{x \ (d \odot d') \ z \quad z \sqsubseteq_{B_3} z'}{x \ (d \odot d') \ z'} \quad \frac{}{\mathcal{U}_{B_1} \ (d \odot d') \ z} \\
\frac{\triangleright_t [\tilde{x}_t \ (d \odot d') \ \tilde{z}_t]}{\theta_{B_1}(\tilde{x}) \ (d \odot d') \ \theta_{B_3}(\tilde{z})}
\end{array}$$

Fig. 6. The free composition of error domain relations.

Again we have a reflexive error domain relation $r(B)$ given by the ordering. The relational composition of two error domain relations in general is not an error domain relation (θ congruence in particular is not preserved), but we instead use an alternative “free composition” $d \odot d' : B \multimap B''$ that is inductively generated from the relational composition. Details of this construction are in the appendix (Definition ??). The rules are given in Figure 6.

We also will use

As in the previous section, we also have the notion of a square $f \sqsubseteq_{c_o}^{c_i} g$ between predomain morphisms as well as, which is defined to mean that for all $x \ c_i \ y$, we have $f(x) \ c_o \ g(y)$. Likewise, we have squares $\phi \sqsubseteq_{d_o}^{d_i} \psi$ between error domain morphisms, which are defined similarly, with no extra interaction with the algebraic structure. The free composition of error domain relations has the universal property that $\phi \sqsubseteq_{d_o}^{d \odot d'} \phi'$ if and only if for any $x \ d \ y$ and $y \ d' \ z$, we have $\phi(x) \ d'' \ \phi'(z)$.

In our denotational model of GTLC, we map every type A to a predomain $\llbracket A \rrbracket$. These constructions are given by the same constructions as in Section 3, but now enriched to act also on orderings and bisimilarity. For products this is the obvious lifting, and for functions we use the CBPV decomposition $U(A \rightarrow L_{\mathcal{U}} A')$ where the only one that is not clear is $L_{\mathcal{U}} A'$.

Definition 5.4. The free error domain $L_{\mathcal{U}} A$ has as the underlying set the free simple error domain with ordering given by the lock-step error ordering and bisimilarity by weak bisimilarity both generated by the ordering/bisimilarity on A . This is free in the same sense as the free simple error domain that it is left adjoint to U .

Additionally, all of these core constructions $\times, \rightarrow, U, L_{\mathcal{U}}$ act on heterogeneous relations as well in an analogous way.

The most interesting type to interpret is the dynamic type. The beauty of guarded domain theory is that once we have define $\triangleright$ for predomains, we can solve the same domain equation as in Section 3 changed only in that we interpret it as a domain equation on predomains rather than sets:

$$D \cong \mathbb{N} + (D \times D) + \triangleright U(D \rightarrow L_{\mathcal{U}} D)$$

The complete definition of the ordering $\sqsubseteq_D$ is provided in the appendix for completeness. We refer to the three injections as e_{Nat} , $e_{\times}$, and $e_{\rightarrow}$, respectively.

To interpret term-type precision, we define three relations involving D as follows. We define $\text{Inj}_{\mathbb{N}} : \mathbb{N} \multimap D$ by $(n, d) \in \text{Inj}_{\mathbb{N}}$ iff $e_{\text{Nat}}(n) \sqsubseteq_D d$. We similarly define $\text{Inj}_{\times} : D \times D \multimap D$ by $((d_1, d_2), d) \in \text{Inj}_{\times}$ iff $e_{\times}(d_1, d_2) \sqsubseteq_D d$, and we define $\text{inj}_{\rightarrow} : U(D \rightarrow L_{\mathcal{U}} D) \multimap D$ by $(f, d) \in \text{inj}_{\rightarrow}$ iff $e_{\rightarrow}(f) \sqsubseteq_D d$ $(e_{\rightarrow}(f) \circ \text{next}) \sqsubseteq_D d$.

5.2 Phase Two: Perturbations and Quasi-Representable Relations

We now introduce the additional definitions needed to complete the construction of our model. Recall from Section 4.3 that in order to establish the DnL-DnR rule for the downcast of $\text{Inj}_\perp$, we needed to adjust the rule by inserting a “delay” on the right-hand side. A similar adjustment is needed for the DnR-DnR rule for $\text{Inj}_\perp$. Moreover, the need to insert delays impacts the semantics of the cast rules for *all* relations, because of the functorial nature of casts. That is, the upcast at a function type $c_i \rightarrow c_o$ involves a downcast in the domain and an upcast in the codomain. This has two consequences: first, the squares corresponding to the rules UpL and UpR , UrL and UrR may also require the insertion of a delay. Second, we need to be able to insert “higher-order” delays in a way that follows the structures of the casts.

To accomplish this, we equip the predomains A in our semantics with a monoid M_A of *syntactic perturbations*, as well as a means of *interpreting* these syntactic perturbations as *semantic perturbations* on A , i.e., as endomorphisms that are bisimilar to the identity. We call such an object a *value object*, and this will serve as the final definition of a denotation of a type in our semantics. We Similarly, we define computation objects to *similarly* be error domains equipped with a notion of syntactic perturbation:

Definition 5.5 (value and computation objects + morphisms). A **value object** consists of a predomain A , a monoid M_A and a homomorphism of monoids $M_A \rightarrow \text{Endo}_\approx(A) := \{f : A \rightarrow A \mid f \approx \text{id}_A\}$ $i_A : M_A \rightarrow \text{Endo}_\approx(A)$.

Likewise, a **computation object** consists of an error domain B , a monoid M_B and a homomorphism of monoids $M_B \rightarrow \text{Endo}_\approx(B)$ $i_B : M_B \rightarrow \text{Endo}_\approx(B)$.

We define a **value morphism** to be *simply* a morphism of the underlying predomains, and a **computation morphism** to be a morphism of the underlying error domains.

Remark 1. Syntactic perturbations are equipped with a monoid structure in order to support identity and composition relations on value and computation types. That is, the identity relation requires an identity perturbation and composition of relations requires a composition operation on perturbations.

Remark 2. It might seem strange that we equip objects with a monoid of perturbations but we do not require morphisms to respect this structure in any way. A natural attempt would be defining a morphism between value objects (A, M_A, i_A) and $(A', M_{A'}, i_{A'})$ to be a predomain morphism $f : A \rightarrow A'$ along with a monoid homomorphism $h : M_A \rightarrow M_{A'}$ such that f and h satisfy a compatibility condition with respect to the interpretations i_A and $i_{A'}$. However, this condition simply fails to hold for all terms, in particular the property is preserved by λ abstraction.

Having defined syntactic perturbations and value and computation objects, we can now make formal the final aspect of the model: the notion of **quasi-representability** quasi-representability of relations.

Definition 5.6 (quasi-left-representable relations). Let (A, M_A, i_A) and $(A', M_{A'}, i_{A'})$ be value objects, and let $c : A \multimap A'$ be a predomain relation. We say that c is *quasi-left-representable* by a predomain morphism $e : A \rightarrow A'$ if there are perturbations $\delta_c^{l,e} \in M_A$ and $\delta_c^{r,e} \in M_{A'}$ such that the following two squares exist: (1) $\text{UpL} : e \sqsubseteq_{\sqsubseteq_{A'}}^c i_{A'}(\delta_c^{r,e})$, and (2) $\text{UpR} : i_A(\delta_c^{l,e}) \sqsubseteq_{\sqsubseteq_A}^c e$.

Observe that this weakens the previous notion of representability, since under that definition the perturbations were required to be the identity. We make the analogous definition of quasi-left-representability for error domain relations d .

Likewise, we define quasi-right-representability as follows:

Definition 5.7 (quasi-right-representable relations). Let (B, M_B, i_B) and $(B', M_{B'}, i_{B'})$ be computation objects, and let $d : B \multimap B'$ be an error domain relation. We say that d is *quasi-right-representable* by an error domain morphism $p : B' \multimap B$ if there are perturbations $\delta_d^{l,p} \in M_B$ and $\delta_d^{r,p} \in M_{B'}$ such that the following two squares exist: (1) DN $\mathcal{L}$: $p \sqsubseteq_{d'}^{B'} i_{B'}(\delta_d^{r,p})$ and (2) DN $\mathcal{R}$: $i_B(\delta_d^{l,p}) \sqsubseteq_B^d p$. Quasi-right-representability for predomain relations is defined similarly.

For ordinary representability, the relation is *uniquely-determined* by any morphism that left- or right-represents it: if two relations are left-represented by the same morphism, then they are equal. But for quasi-representability, this is no longer the case due to the presence of perturbations. Instead we have the following weaker property:

Definition 5.8 (quasi-equivalence). Let A and A' be value objects, and let $c, c' : A \multimap A'$ be relations between the underlying predomains. We say that c and c' are **quasi-equivalent**, written $c \approx c'$, if there exist syntactic perturbations $\delta_1^l, \delta_2^l \in M_A$ and $\delta_1^r, \delta_2^r \in M_{A'}$ such that there is a square $i_A(\delta_1^l) \sqsubseteq_{c'}^c i_{A'}(\delta_1^r)$ and a square $i_A(\delta_2^l) \sqsubseteq_{c'}^{c'} i_{A'}(\delta_2^r)$. Given computation objects B and B' , we make the analogous definition for relations $d, d' : B \multimap B'$ between the underlying error domains.

Given two quasi-representable relations c and c' , it is not the case in general that cc' is quasi-representable. We need an additional condition that specifies how relations interact with perturbations. In particular, we need to be able to “push” and “pull” perturbations along relations c and d . The intuition for this requirement comes from the construction of the square corresponding to the original $\text{UpL-}\mathcal{U}\mathcal{P}\mathcal{L}$ rule from the square for the simplified version as was shown in Section 4. Recall that we horizontally composed the simplified $\text{UpL-}\mathcal{U}\mathcal{P}\mathcal{L}$ square for c with the identity square for c' on the right. Now that the square for $\text{UpL-}\mathcal{U}\mathcal{P}\mathcal{L}$ involves a perturbation on the right instead of the identity, this construction will not work unless we can “push” the perturbation on A_2 to one on A_3 . That is, given a relation $c : A \multimap A'$ and a syntactic perturbation $m_A \in M_A$, we need to be able to turn it into a syntactic perturbation $m_{A'} \in M_{A'}$ such that the resulting semantic perturbations obtained by applying the respective homomorphisms i_A and $i_{A'}$ form a square. Dually, we will need to “pull” a perturbation from right to left. This notion is made formal by the following definition:

Definition 5.9. Let (A, M_A, i_A) and $(A', M_{A'}, i_{A'})$ be value objects, and let $c : A \multimap A'$ be a predomain relation. A **push-pull structure** for c (with respect to M_A and $M_{A'}$) consists of monoid homomorphisms $\text{push} : M_A \rightarrow M_{A'}$ and $\text{pull} : M_{A'} \rightarrow M_A$ such that for all $m_A \in M_A$ there is a square $i_A(m_A) \sqsubseteq_{c'}^c i_{A'}(\text{push } m_A)$ and for all $m_{A'} \in M_{A'}$ there is a square $i_A(\text{pull } m_{A'}) \sqsubseteq_c^{c'} i_{A'}(m_{A'})$. A push-pull structure for error domain relations is defined similarly.

We can now give the final definition of value and computation relations for our model, which serve as the denotation of our type precision derivations:

Definition 5.10 (value relations). A **value relation** between value objects (A, M_A, i_A) and $(A', M_{A'}, i_{A'})$ consists of a predomain relation $c : A \multimap A'$ that has a push-pull structure, is quasi-left-representable by a morphism $e_c : A \rightarrow A'$, and is such that $L_{\mathcal{U}}c$ is quasi-right-representable by an error domain morphism $p_c : L_{\mathcal{U}}A' \rightarrow L_{\mathcal{U}}A$.

A **computation relation** between computation objects (B, M_B, i_B) and $(B', M_{B'}, i_{B'})$ consists of an error domain relation d that has a push-pull structure, is quasi-right-representable by a morphism $p_d : B' \rightarrow B$, and is such that Ud is quasi-left-representable by a morphism $e_d : UB \rightarrow UB'$.

It is essential in the definition of a value relation that we have not just that c is quasi-left-representable but also that $L_{\mathcal{U}}c$ is quasi-right-representable so that we can define the action of the functor $L_{\mathcal{U}}$ on relations, taking a value relation to a computation relation. Likewise, we require Ud

to be quasi-left-representable in the definition of computation relations so that we can define the action of U on relations.

To model the term precision in a way that is oblivious to the stepping behavior of terms, we need to weaken the notion of square to allow for the morphisms on either side to be “out of sync” rather than in lock step. In the below, $f' \approx_{A_o}^{A_i} f$ is the natural extension of bisimilarity to morphisms: given bisimilar inputs, the functions have bisimilar outputs.

Definition 5.11. Let $c_i : A_i \multimap A'_i$, $c_o : A_o \multimap A'_o$, $f : A_i \rightarrow A_o$ and $g : A'_i \rightarrow A'_o$. An extensional square $f \sqsubseteq_{c_o}^{c_i} g$ holds if there exist $f' \approx_{A_o}^{A_i} f$ and $g' \approx_{A'_o}^{A'_i} g$ such that $f' \sqsubseteq_{c_o}^{c_i} g'$.

~~Here $f' \approx_{A_o}^{A_i} f$ is the natural extension of bisimilarity to morphisms: given bisimilar inputs the functions have bisimilar outputs.~~

5.3 Constructing the ~~denotational semantics~~ Denotational Semantics

With these concepts now defined, we can give an overview of our denotational model:

- For every type A we define a value type $\llbracket A \rrbracket$, and define $\llbracket x_1 : A_1, \dots \rrbracket = \llbracket A_1 \rrbracket \times \dots$
- For every term $\Gamma \vdash M : A$, we define a value morphism $\llbracket M \rrbracket : \llbracket \Gamma \rrbracket \rightarrow UL_U \llbracket A \rrbracket$.
- If $M = M'$ are equal according to our equational theory, then $\llbracket M \rrbracket = \llbracket M' \rrbracket$
- For every type precision derivation $c : A \sqsubseteq A'$ we define a value relation $\llbracket c \rrbracket : \llbracket A \rrbracket \multimap \llbracket A' \rrbracket$.
- For every type precision equivalence $c \equiv c'$ we have a ~~quasi-order-equivalence~~ quasi-equivalence of relations $\llbracket c \rrbracket \approx \llbracket c' \rrbracket$.
- For every term precision $\Delta \vdash M \sqsubseteq M' : c$ we have an *extensional* square $\llbracket M \rrbracket \sqsubseteq_{UL_U \llbracket c \rrbracket}^{\llbracket \Delta \rrbracket} \llbracket M' \rrbracket$

~~For the The term semantics $\llbracket M \rrbracket$, nothing really differs from is nearly identical to the term semantics in Section 3. The only One difference is that the definitions of $\llbracket \text{up } e \rrbracket$ and $\llbracket \text{dn } e \rrbracket$ can be defined as the morphisms that left- and right-quasi-represent $\llbracket e \rrbracket$, but upon inspection this definition agrees with the one given before. Similarly, the interpretation of the function type $A_i \rightarrow A_o$ in the term semantics of Section 3 involves arbitrary functions, while the interpretation of the function type in the term semantics defined here involves only those functions that are morphisms of predomains. Additionally, the semantics of casts is slightly different. In the semantics of Section 3, the projection corresponding to the type precision cc' is simply the composition of the projections. In the new semantics, the corresponding projection involves a perturbation. This is a consequence of the construction of the composition of value/computation relations (the relevant projection is defined in the proof of Lemma ?? in the Appendix). However, the two projections are extensionally equivalent since they are the same up to a perturbation.~~

~~The validity of the equational theory follows easily: $\beta\eta$ holds in our model. We next In the remainder of this section, we elaborate on each of the remaining components of the model.~~

5.3.1 Interpreting types. For our value type semantics, we already have an interpretation as predomains; we describe how we enhance each of the constructions we use ($U, L_U, \rightarrow, \times, D$ ~~with a notion~~) with a monoid of syntactic perturbation and a means of interpreting them as semantic perturbations.

First consider the free error domain L_U- . Given a value object (A, M_A, i_A) we define the syntactic perturbations $M_{L_U A}$ to be $\mathbb{N} \oplus M_A$, where $\oplus$ denotes the free product of monoids (the coproduct in the category of monoids). The intuition behind needing to include $\mathbb{N}$ comes from the example of the downcast for $\text{Inj}_\rightarrow$. Recall that we adjusted the squares corresponding to the ~~DnL and DnR~~ DnL and DnR rules by adding $(\delta \circ \eta)^\dagger$ on the side opposite the downcast. Thus, we need a syntactic perturbation in $M_{L_U A}$ that will be interpreted as $(\delta \circ \eta)^\dagger$. We accomplish this by taking

the coproduct with $\mathbb{N}$. Since $\mathbb{N}$ is the free monoid on one generator, defining a homomorphism from $\mathbb{N}$ to $\text{Endo}_{\approx}(L_{\mathcal{U}}A)$ is equivalent to a choice of an element in $\text{Endo}_{\approx}(L_{\mathcal{U}}A)$. The interpretation of the perturbations as endomorphisms is defined using the universal property of the coproduct of monoids (see appendix for details). Specifically, to define the homomorphism $i_{L_{\mathcal{U}}A}$, by the universal property of the coproduct of monoids it suffices to define two homomorphisms, one from $\mathbb{N} \rightarrow \text{Endo}_{\approx}(L_{\mathcal{U}}A)$ and one from $M_A \rightarrow \text{Endo}_{\approx}(L_{\mathcal{U}}A)$. The first homomorphism sends the generator $1 \in \mathbb{N}$ to the error domain endomorphism $(\delta \circ \eta)^\dagger$, and the second sends $m_A \in M_A$ to $L_{\mathcal{U}}(i_A(m_A))$. We observe in both cases that the resulting endomorphisms are bisimilar to the identity, since in the first case, $-^\dagger$ preserves bisimilarity, and in the second case, the action of $L_{\mathcal{U}}$ on morphisms preserves bisimilarity.

Now we discuss define the action of U on objects. Given a computation object (B, M_B, i_B) , we define M_{UB} to be $\mathbb{N} \oplus M_B$. The reason for requiring $\mathbb{N}$ is related to the Kleisli action of the arrow functor: to establish quasi-representability of $U(c \rightarrow d)$ given quasi-representability of c and d , we will need to turn a perturbation on $L_{\mathcal{U}}A$ into a perturbation on $U(A \rightarrow B)$. Since the perturbations for $L_{\mathcal{U}}A$ involve $\mathbb{N}$, so must the perturbations for UB . The interpretation i_{UB} of the perturbations on UB works in the same manner as an analogous manner to that of $L_{\mathcal{U}}A$: we send the generator $1 \in \mathbb{N}$ to the delay morphism $\delta_B: UB \rightarrow UB$, and we send a perturbation $m_B \in M_B$ to $U(i_B(m_B))$.

The action of $\times$ on objects is as follows. Given (A_1, M_{A_1}, i_{A_1}) and (A_2, M_{A_2}, i_{A_2}) , we define the monoid $M_{A_1 \times A_2} = M_{A_1} \oplus M_{A_2}$. The interpretation $i_{A_1 \times A_2}$ is defined using the universal property of the coproduct (see appendix for the details) of monoids. That is, we define homomorphisms $M_{A_1} \rightarrow \text{Endo}_{\approx}(A_1 \times A_2)$ and $M_{A_2} \rightarrow \text{Endo}_{\approx}(A_1 \times A_2)$. The former takes $m_1 \in M_{A_1}$ to the morphism $(i_{A_1}(m_1), \text{id})$, while the latter takes $m_2 \in M_{A_2}$ to the morphism $(\text{id}, i_{A_2}(m_2))$.

The action of $\rightarrow$ on objects is as follows. Given (A, M_A, i_A) and (B, M_B, i_B) we define the monoid $M_{A \rightarrow B} = M_A^{op} \oplus M_B$ and the interpretation is defined analogously to that of $\times$. (Here, M^{op} denotes the monoid with the same elements but with the order of multiplication reversed, i.e., $x \cdot^{M^{op}} y := y \cdot^M x$.) To define the interpretation $i_{A \rightarrow B}$, it suffices to define a homomorphism $M_A^{op} \rightarrow \text{Endo}_{\approx}(A \rightarrow B)$ and $M_B \rightarrow \text{Endo}_{\approx}(A \rightarrow B)$. The former takes an element m_A and a predomain morphism $f: A \rightarrow UB$ and returns the composition $f \circ i_A(m_A)$, while the latter is defined similarly by post-composition with i_B .

Finally, the most interesting type is of course the dynamic type. Now we define the perturbations for D . Recall that to each predomain A we associate a monoid M_A of syntactic perturbations and a homomorphism into the monoid of endomorphisms bisimilar to the identity, and likewise for error domains. It may seem as though we need to define the perturbations for D by guarded recursion, but in fact we define them via least-fixpoint in the category of monoids: $M_D \cong (M_{D \times D}) \oplus M_{U(D \rightarrow L_{\mathcal{U}}D)}$.

Remark 3. Here it is crucial to define the perturbations of the dynamic type inductively rather than by guarded recursion. If we were to define the perturbations by guarded recursion, then it would be impossible to define the pull homomorphism for the $\text{Inj}_{\rightarrow}$ relation.

We now explain how to interpret these perturbations as endomorphisms.

We define $i_D: M_D \rightarrow \{f: D \rightarrow D \mid f \approx \text{id}\}$ by induction as follows:

$$i_D(m_{\times}) = \lambda d. \text{case } d \text{ of } \{\text{Inj}_{\times}(d_1, d_2) \Rightarrow \text{Inj}_{\times}(i_{D \times D}(m_{\times})(d_1, d_2)) \mid x \Rightarrow x\}$$

$$i_D(m_{\rightarrow}) = \lambda d. \text{case } d \text{ of } \{\text{Inj}_{\rightarrow}(\tilde{f}) \Rightarrow \text{Inj}_{\rightarrow}(\lambda t. i_{U(D \rightarrow L_{\mathcal{U}}D)}(m_{\rightarrow})(\tilde{f}_t)) \mid x \Rightarrow x\}$$

In the case of a perturbation on $D \times D$, we use the interpretation for the perturbations of $D \times D$ as defined earlier in this section, which in turn will use i_D inductively. Similarly, for a perturbation on $U(D \rightarrow L_{\mathcal{U}}D)$ we use the interpretation for perturbations on $U(D \rightarrow L_{\mathcal{U}}D)$. One can verify that this defines a homomorphism from $M_D \rightarrow \{f: D \rightarrow D : f \approx \text{id}\}$.

5.3.2 *Interpreting type precision.* Next we need to confirm that our interpretation of type precision derivations as predomain relations indeed extends to value relations, i.e., that the relations are quasi-representable and that they satisfy the push-pull property for perturbations. We include most of these technical verifications in the appendix (Definition ??, Definitions ?? and ??).

We go into more detail on the three relations involving the dynamic type $\text{Inj}_{\mathbb{N}}$, $\text{Inj}_{\times}$, and $\text{Inj}_{\rightarrow}$. As an illustrative case, we establish the push-pull property for the relation $\text{Inj}_{\rightarrow}$. We define $\text{pull}_{\text{Inj}_{\rightarrow}} : M_D \rightarrow M_{U(D \rightarrow L_U D)}$ by giving a homomorphism from $M_{D \times D} \rightarrow M_{U(D \rightarrow L_U D)}$ and from $M_{U(D \rightarrow L_U D)} \rightarrow M_{U(D \rightarrow L_U D)}$. The former is the trivial homomorphism sending everything to the identity element, while the latter is the identity homomorphism. Conversely, we define $\text{push}_{\text{Inj}_{\rightarrow}} : M_{U(D \rightarrow L_U D)} \rightarrow M_D$ as the inclusion into the coproduct. Showing that the relevant squares exist involving push and pull is straightforward.

It is easy to see that the relations $\text{Inj}_{\mathbb{N}}$, $\text{Inj}_{\times}$, and $\text{Inj}_{\rightarrow}$ are quasi-left-representable, where all perturbations are taken to be the identity elements of the respective monoids. For quasi-right-representability, the only nontrivial case is $L_U(\text{Inj}_{\rightarrow})$. Recall the definition of the downcast for $\text{Inj}_{\rightarrow}$ given in Section 3.4. We know that in order for the DnR and DnL squares for $L_U \text{Inj}_{\rightarrow}$ to exist, we must insert a delay on the side opposite the downcast. In terms of syntactic perturbations, this means that we take $\delta^{l,p}$ and $\delta^{r,p}$ in the definition of quasi-right-representability to both be $\text{inl } 1$, where inl is the left injection into the coproduct of monoids. We recall that the definition of syntactic perturbations for L_U involves a coproduct with $\mathbb{N}$, and that the interpretation homomorphism maps the generator 1 to the endomorphism of error domains $(\delta \circ \eta)^{\dagger}$. With this choice for the perturbations, it is straightforward to show that the DnL and DnR squares exist using the definition of the downcast and the definition of the relation $\text{Inj}_{\rightarrow}$.

The proofs that the precision equivalence imply $c \equiv c'$ as quasi-equivalence of relations are in the appendix (Lemma ??) the denoted relations $\llbracket c \rrbracket \approx \llbracket c' \rrbracket$ is justified by Lemma ?? in the Appendix.

5.3.3 *Interpreting Term Precision: Extensional Squares.* Finally, we establish the core result of the semantics: interpreting term precision as extensional squares. Fortunately, the hard work is now complete, and the term precision rules follow from our compositional constructions. The In particular, the congruence rules of term precision are modeled easily: congruence squares exist both for bisimilarity as well as and for the error ordering, and so by combination we also have these for extensional squares. The type precision equivalence rule ERRBOT rule holds because $\mathbb{U}$ is the least element in the semantics of the. Next, the EQUIVTPREC rule follows because $c \equiv c'$ implies quasi-equivalence, and that quasi-equivalence of the denoted relations, and this in turn implies that $\text{id} \sqsubseteq_{\llbracket c' \rrbracket}^{\llbracket c \rrbracket} \text{id}$.

Most crucially, we Lastly, it remains to verify the existence of the squares for the cast rules. For UpLUpL , the relevant extensional square is obtained as follows:

$$\begin{array}{ccccc}
 A_1 & \xrightarrow{c} & A_2 & \xrightarrow{c'} & A_3 \\
 e_c \left(\approx \right) e_c \downarrow & & \delta_c^{r,e} \downarrow & \text{push}_{c'}(\delta_c^{r,e}) \left(\approx \right) \downarrow & \text{id} \\
 A_2 & \xrightarrow{r(A_2)} & A_2 & \xrightarrow{c'} & A_3
 \end{array}$$

We have used the push-pull structure on c' to turn the perturbation on A_2 into a perturbation on A_3 . Lastly we note that the bottom edge of the square is equal to c' by the fact that c' is downward closed. The squares for UpR , DnL , and DnR are obtained analogously.

5.4 Relational Adequacy

Now that we have established our denotational model, we can, as in Section 3.5, extract a big step semantics. Our final goal then is to prove that the extensional squares in our model imply the graduality property for this final big step semantics. The result we seek to show is the following:

THEOREM 5.12 (ADEQUACY). *If $\cdot \vdash M \sqsubseteq N : \text{Nat}$, then:*

- *If $M \Downarrow n$ then $N \Downarrow n$.*
- *If $N \Downarrow \mathcal{U}$ then $M \Downarrow \mathcal{U}$.*
- *If $N \Downarrow n$ then $M \Downarrow n$ or $M \Downarrow \mathcal{U}$.*

It therefore remains to show that the denotation of term precision in our model $f \sqsubseteq g$, where $f, g : L_{\mathcal{U}}\mathbb{N}$, implies a relationship between the partial elements $\mathbb{N} + \mathcal{U}$ corresponding to f and g . Recall that to define the big-step semantics we used the equivalence of the globalization of the free error domain and Capretta's coinductive delay monad: $L_{\mathcal{U}}^g\mathbb{N} \cong \text{Delay}(\mathbb{N} + \mathcal{U})$. We now extend this correspondence to a relationship between a global version of our denotation of term precision and the corresponding relation on the delay monad.

To that end, we define, using clock quantification, a version of our denotation of term precision that relates terms of type $L_{\mathcal{U}}^g\mathbb{N}$. More concretely, consider the predomain $\mathbb{N}$ with ordering and bisimilarity both the equality relation. We define a global version of the lock-step error ordering and the weak bisimilarity relation on elements of $L_{\mathcal{U}}^g\mathbb{N}$; the former is defined by $x \sqsubseteq^{gl} y := \forall k. x[k] \sqsubseteq y[k]$, and the latter is defined by $x \approx^{gl} y := \forall k. x[k] \approx y[k]$. In these two definitions, we apply the clock k to the elements x and y of type $L_{\mathcal{U}}^g\mathbb{N} := \forall (k : \text{Clock}). L_{\mathcal{U}}^k\mathbb{N}$.

Correspondingly, we define a version of the lock-step error ordering and weak bisimilarity relation on $\text{Delay}(\mathbb{N} + \mathcal{U})$ (see Section ?? in the appendix for the definitions). By adapting Theorem 4.3 of [5] to the setting of inductively-defined relations, we can show that both the global lock-step error ordering and the global weak bisimilarity admit coinductive definitions. In particular, modulo the above isomorphism between $L_{\mathcal{U}}^g\mathbb{N}$ and $\text{Delay}(\mathbb{N} + \mathcal{U})$, the global version of the lock-step error ordering is equivalent to the lock-step error ordering on $\text{Delay}(\mathbb{N} + \mathcal{U})$, and the global version of weak bisimilarity is equivalent to weak bisimilarity on $\text{Delay}(\mathbb{N} + \mathcal{U})$. This implies that the global denotation of the term precision semantics for $L_{\mathcal{U}}^g\mathbb{N}$ agrees with the corresponding relation $\sqsubseteq^{\text{Delay}}$ for $\text{Delay}(\mathbb{N} + \mathcal{U})$, i.e., the closure of its lock-step ordering under weak bisimilarity on both sides. Then to obtain the desired result, it suffices to show being related in $\sqsubseteq^{\text{Delay}}$ implies that the corresponding partial elements of $\mathbb{N} + \mathcal{U}$ are related in the expected manner. That is, if $d \sqsubseteq^{\text{Delay}} d'$, then $p(d) \sqsubseteq^{\text{Partial}} p(d')$ where p is a partial function from delays into $\mathbb{N} + \mathcal{U}$ defined by existential quantification over the number of steps i in which d terminates, and the relation $\sqsubseteq^{\text{Partial}}$ is defined such that $\mathcal{U}$ is the least element, $\perp \sqsubseteq^{\text{Partial}} \perp$, and $n \sqsubseteq^{\text{Partial}} n$. This implication follows from the definition of $\sqsubseteq^{\text{Delay}}$.

6 DISCUSSION

6.1 Related Work

We give an overview of current approaches to proving graduality and discuss their limitations where appropriate.

The methods of Abstracting Gradual Typing [14] and the formal tools of the Gradualizer [9] provide a way to derive from a statically-typed language a language that satisfies graduality by construction. The main downside to these approaches lies in their inflexibility: since the process is entirely mechanical, the language designer must adhere to the predefined framework. Many

gradually typed languages do not fit into either framework, e.g., Typed Racket [37, 38], and the semantics produced is not always the desired one. Furthermore, while these frameworks do prove graduality of the resulting languages, they do not show the correctness of the equational theory, which is equally important to sound gradual typing.

New and Licata [29] developed an axiomatic account of the graduality relation on a call-by-name λ -calculus and showed that the graduality proof could be modeled using semantics in certain kinds of double categories. Later, New, Licata and Ahmed [30] extended this axiomatic treatment from call-by-name to call-by-value as well by giving an axiomatic theory of type and term precision in call-by-push-value. The double-categorical framework serves as a foundation on which we base our model construction in this paper. However, because of the lack of transitivity of the relation modeling term precision, our model is not a double category but instead a reflexive graph category, that is, a double category without a notion of relational composition.

A line of work by New, Licata and Ahmed [26, 29, 30] develops an axiomatic account of gradual typing and proves graduality by interpreting type precision $c : A \sqsubseteq A'$ as an *embedding-projection pair*, that is a *pure*, a *pure embedding* function $e : A \rightarrow A'$ and a possibly erroring projection $p : L_{\cup}A' \multimap L_{\cup}A$ such that $p \circ e = \text{id}$ and $e \circ p \sqsubseteq \text{id}$ [26, 29, 30]. At first glance, our model of precision looks different: it is a *relation* $\llbracket c \rrbracket : A \multimap A'$ with a quasi-representability structure that gives us something like *close to* e and p in an embedding-projection pair. The relationship between these models is that if the relation were *truly* representable we would have that e and p form a *Galois connection* $e \circ p \sqsubseteq \text{id}$ and $\text{id} \sqsubseteq p \circ e$. However, we have dropped the stronger property of retraction from our analysis in this work. With true representability, the relation c is *uniquely determined* by the embedding e , but since we only have quasi-representability, we need to keep the relation around explicitly. So the extra complexity of managing explicit relations is a cost of the intensional reasoning that guarded type theory introduces. The line of work by New, Licata, and Ahmed involved both denotational methods and operational logical relations approaches.

~~The standard alternative to domain theory that scales to essentially arbitrary~~

On the denotational side, New and Licata [29] showed that the graduality proof could be modeled using semantics in certain kinds of double categories, which are categories internal to the category of categories. A double category extends a category with a second notion of morphism, often a notion of “relation” to be paired with the notion of functional morphism, as well as a notion of functional morphisms preserving relations. In gradual typing, the notion of relation models type precision and the squares model the term precision relation. The double-categorical framework serves as a foundation on which we base our model construction in this paper. However, our model is not actually a double category: the lack of transitivity of bisimilarity means that we cannot compose extensional squares horizontally.

With the notion of abstract categorical model for gradual typing in hand, denotational semantics is then the work of constructing concrete models that exhibit the categorical construction. New and Licata [29] present such a model using categories of ω -CPOs, and this model was extended by Lennon-Bertrand, Maillard, Tabareau and Tanter [17] to prove graduality of a gradual dependently typed calculus CastCIC^G . This domain-theoretic approach meets our criterion of being a semantic framework for proving graduality, but suffers from the limitations of classical domain theory: the inability to model viciously self-referential definitions is *step-indexing* or its synthetic form of *guarded recursion*. A structures such as higher-order extensible state and similar features such as *runtime-extensible dynamic types*.

On the operational side, a series of works [26, 28, 30] developed step-indexed logical relations models of gradually typed languages based on operational semantics. Unlike classical domain theory, such step-indexed techniques are capable of modeling can scale to essentially arbitrary

self-referential definitions, which means they can model higher-order store and runtime-extensible dynamic types [1, 2, 25, 28]. However, as is common with operational approaches, their proof developments are highly repetitive and technical, with each development formulating a logical relation from first-principles and proving many of the same tedious lemmas without reusable mathematical abstractions. This is addressed somewhat by Siek and Chen [32], who give a proof in Agda of graduality for an operational semantics ~~who use using~~ a *guarded logic of propositions* shallowly embedded in Agda. The guarded logic simplifies the treatment of step-indexed logical relations, but the approach is still fundamentally operational, and so the main lemmas of the work are still tied to the particular operational syntactic calculus being modeled. ~~A further advantage of the denotational approach is that it easily validates equational reasoning, not just graduality, and it is completely independent of any particular syntax of gradual typing.~~

Our work combines the benefits of both the denotational and step-indexed approaches. We take as a starting point the denotational approach of New and Licata, but we work with guarded type theory (a synthetic form of step-indexing) rather than classical domain theory. As we have seen, in the guarded setting the compositional theory of New-Licata breaks down. Our work provides a novel refinement to their theory that allows for some compositional reasoning to be recovered even in the guarded setting.

Eremondi [12] uses guarded type theory to define a syntactic model for a gradually-typed source language with dependent types. By working in guarded type theory, they are able to give an exact semantics to non-terminating computations in a language which is logically consistent, allowing for metatheoretic results about the source language to be proven. Similarly to our approach, they define a guarded lift monad to model potentially- nonterminating computations and use guarded recursion to model the dynamic type. However, they do not give a denotational semantics to term precision and it is unclear how to prove graduality in this setting. ~~The~~ Their work includes a formalization of the syntactic model in Guarded Cubical Agda.

6.2 Comparison of Denotational and Operational Approaches

Denotational methods have several benefits compared to operational approaches, and this carries over to the setting of gradual typing. First, denotational methods are compositional and reusable, and allow for the use of existing mathematical constructs and theorems, e.g., partial orderings, monads, etc, while operational methods tend to require a significant amount of boilerplate work to be done from scratch in each new development. As a specific example of the compositional nature of our approach, the treatment of the cast rules in our work is more compositional than in previous work using operational semantics. Recall from Section 4 that the cast rules needed for the proof of graduality build in composition of type precision derivations. Rather than proving these from scratch in the model, we are able to take as primitive simpler versions of the cast rules whose validity in the model is easier to establish. Then from these simpler rules, we derive the original ones using compositional reasoning.

A further advantage of the denotational approach is that it is completely independent of any particular syntax of gradual typing. This allows for reuse across multiple languages and makes it particularly straightforward to accommodate additions to a language: adding support for a new type amounts to defining a new object and extending the dynamic type accordingly. It also may be possible to model alternative cast semantics by changing the definition of the dynamic type and some type casts. In contrast, operational methods are not as readily extensible, generally requiring adding cases to the logical relations and the inductive proofs. In a mechanized metatheory, this manifests as a “copy-paste” reusability rather than the true code reuse one obtains with a denotational semantics. An additional benefit to the denotational approach is that it is trivial to establish the

validity of the β and η principles, because they are equalities in the semantics, whereas they require tedious manual proof in the operational approach.

Finally, while we advocate for a denotational approach, the overall structure of our semantics could be applied to an operational logical relations proof. For example, there could be a step-indexed logical relation for strong error ordering and one for weak bisimilarity, corresponding to the fact that objects in our denotational model have an error ordering and a bisimilarity relation. Then the precision rules for casts could be similarly validated using perturbations as we did in this work.

6.3 Mechanization

In parallel with developing the theory discussed in this paper, we have developed a partial formalization of our results in Guarded Cubical Agda [39]. We formalized the major components of the definition of the concrete model described in Section 5: predomains, error domains, morphisms, relations, squares, using the guarded features to define the free error domain, the lock-step error ordering, and weak bisimilarity. We also formalized the no-go theorem from Section 3.4.

Building on these definitions, we formalized the notion of semantic perturbations and push-pull structures as well as quasi-representable relations, culminating in the definition of value and computation types and relations as introduced in Section 5. We constructed the predomain for the dynamic type via mixed induction and guarded recursion, and defined its monoid of perturbations. We also defined the relations $\text{Inj}_{\mathbb{N}}$, $\text{Inj}_{\times}$, and $\text{Inj}_{\rightarrow}$, and proved that they are quasi-representable. ~~We have not yet formalized the constructions involving value and computation relations: showing that these relations compose, and defining the actions of the functors $L_{\mathbb{O}}$, U , $\times$, and $\rightarrow$ on the relations. We have also not yet formalized the syntax-to-semantics translation.~~

Lastly, we have formalized the big-step term semantics discussed in Section 3.5 and the adequacy of the relational model discussed in Section 5.4. This required us to add axioms about clock quantification as well as axioms asserting the *clock-irrelevance* of booleans and natural numbers since as of this writing these axioms are not built-in to Guarded Cubical Agda. These axioms are discussed in prior work on guarded type theory [3, 5]. The mechanization of adequacy also required us to formalize some essential lemmas involving clocks and clock-irrelevance; we are considering later refactoring these as part of a “standard library” for Guarded Cubical Agda.

The remaining formalization work is the following:

- (1) Showing that the functors $\times$ and $\rightarrow$ preserve quasi-representability, which requires tedious reasoning about the Kleisli actions of $\times$ and $\rightarrow$. These results are proved in the Appendix.
- (2) Verifying the rules in the model corresponding to the equations for type precision derivations (see the bottom of Figure 1). These are also included in the Appendix.
- (3) Formalizing the syntax-to-semantics translation.

6.4 Future Work

~~In the future, we plan~~ Our immediate next step is to apply our approach to give a denotational semantics ~~for languages that feature to~~ gradually-typed languages with advanced type systems including higher-order state or runtime-extensible dynamic typing [36], as well as richer type disciplines such as gradual dependent types and effect systems. This will involve adapting prior operational models based on step-indexing (e.g., [27]) to the denotational setting and using guarded type theory to obtain solutions to guarded domain equations as we did in this work for the denotation of the dynamic type. The generality of our techniques should allow for many of the constructions to be reused across different languages. For instance, the free error domain construction can be easily extended ~~with additional cases~~ to model effects besides error and stepping ~~–Additionally, we~~

[13, 35]. We also aim to complete our Agda formalization and evolve it into a reusable framework for mechanized denotational semantics of gradually-typed languages.

This work has focused on the “Natural” semantics of casts, which validate the full call-by-value η laws for types, but other cast semantics have been proposed such as eager ([16]) and transient ([40]) cast semantics which trade off certain η equalities for other benefits such as early error detection or reduced runtime overhead [15, 30]. It would be a good test of the generality of our semantic framework to see if it can model these alternative semantics by adjusting the semantics of types and casts, and providing proofs of weakened η principles.

Additionally, while our focus in this work has been on reasoning up to weak bisimilarity, the presence of explicit step counting in the model could be viewed as a form of *cost semantics*, where a runtime cost is incurred from inspecting the dynamic type. This could possibly be used to verify that cast optimizations such as space efficient implementations [16] are not only extensionally correct, but have a lower abstract cost.

REFERENCES

- [1] Amal J. Ahmed. 2006. Step-Indexed Syntactic Logical Relations for Recursive and Quantified Types. In *Programming Languages and Systems, 15th European Symposium on Programming, ESOP 2006, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2006, Vienna, Austria, March 27-28, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3924)*, Peter Sestoft (Ed.). Springer, 69–83. https://doi.org/10.1007/11693024_6
- [2] Andrew W. Appel and David McAllester. 2001. An indexed model of recursive types for foundational proof-carrying code. *ACM Trans. Program. Lang. Syst.* 23, 5 (sep 2001), 657–683. <https://doi.org/10.1145/504709.504712>
- [3] Robert Atkey and Conor McBride. 2013. Productive Coprogramming with Guarded Recursion. In *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming (Boston, Massachusetts, USA) (ICFP ’13)*. Association for Computing Machinery, New York, NY, USA, 197–208. <https://doi.org/10.1145/2500365.2500597>
- [4] Patrick Bahr, Hans Bugge Grathwohl, and Rasmus Ejlers Møgelberg. 2017. The clocks are ticking: No more delays!. In *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 1–12. <https://doi.org/10.1109/LICS.2017.8005097>
- [5] Magnus Baunsgaard Kristensen, Rasmus Ejlers Mogelberg, and Andrea Vezzosi. 2022. Greatest HITS: Higher Inductive Types in Coinductive Definitions via Induction under Clocks. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (Haifa, Israel) (LICS ’22)*. Association for Computing Machinery, New York, NY, USA, Article 42, 13 pages. <https://doi.org/10.1145/3531130.3533359>
- [6] Lars Birkedal, Rasmus Ejlers Mogelberg, Jan Schwinghammer, and Kristian Støvring. 2011. First Steps in Synthetic Guarded Domain Theory: Step-Indexing in the Topos of Trees. In *2011 IEEE 26th Annual Symposium on Logic in Computer Science*. 55–64. <https://doi.org/10.1109/LICS.2011.16>
- [7] Lars Birkedal, Kristian Støvring, and Jacob Thamsborg. 2009. Realizability Semantics of Parametric Polymorphism, General References, and Recursive Types. In *Foundations of Software Science and Computational Structures*, Luca de Alfaro (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 456–470.
- [8] Venanzio Capretta. 2005. General Recursion via Coinductive Types. *Logical Methods in Computer Science* Volume 1, Issue 2 (July 2005). [https://doi.org/10.2168/LMCS-1\(2:1\)2005](https://doi.org/10.2168/LMCS-1(2:1)2005)
- [9] Matteo Cimini and Jeremy G. Siek. 2016. The Gradualizer: A Methodology and Algorithm for Generating Gradual Type Systems. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (St. Petersburg, FL, USA) (POPL ’16)*. Association for Computing Machinery, New York, NY, USA, 443–455. <https://doi.org/10.1145/2837614.2837632>
- [10] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. 2017. Cubical type theory: A constructive interpretation of the univalence axiom. *IFCoLog J. Log. Appl.* (2017).
- [11] Brian Patrick Dunphy. 2002. *Parametricity As a Notion of Uniformity in Reflexive Graphs*. Ph. D. Dissertation. University of Illinois at Urbana-Champaign, Champaign, IL, USA. Advisor(s) Reddy, Uday.
- [12] Joseph S. Eremondi. 2023. *On the design of a gradual dependently typed language for programming*. Ph. D. Dissertation. University of British Columbia. <https://doi.org/10.14288/1.0428823>
- [13] Dan Frumin, Amin Timany, and Lars Birkedal. 2024. Modular Denotational Semantics for Effects with Guarded Interaction Trees. *Proc. ACM Program. Lang.* 8, POPL, Article 12 (Jan. 2024), 30 pages. <https://doi.org/10.1145/3632854>
- [14] Ronald Garcia, Alison M. Clark, and Éric Tanter. 2016. Abstracting Gradual Typing. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (St. Petersburg, FL, USA) (POPL ’16)*. Association for Computing Machinery, New York, NY, USA, 429–442. <https://doi.org/10.1145/2837614.2837670>

- [15] Ben Greenman. 2022. Deep and shallow types for gradual languages. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation* (San Diego, CA, USA) (PLDI 2022). Association for Computing Machinery, New York, NY, USA, 580–593. <https://doi.org/10.1145/3519939.3523430>
- [16] David Herman, Aaron Tomb, and Cormac Flanagan. 2010. Space-efficient gradual typing. *Higher Order Symbol. Comput.* 23, 2 (June 2010), 167–189. <https://doi.org/10.1007/s10990-011-9066-z>
- [17] Meven Lennon-Bertrand, Kenji Maillard, Nicolas Tabareau, and Éric Tanter. 2022. Gradualizing the Calculus of Inductive Constructions. *ACM Trans. Program. Lang. Syst.* 44, 2, Article 7 (apr 2022), 82 pages. <https://doi.org/10.1145/3495528>
- [18] Paul Blain Levy. 1999. Call-by-Push-Value: A Subsuming Paradigm. In *Typed Lambda Calculi and Applications, 4th International Conference, TLCA'99, L'Aquila, Italy, April 7-9, 1999, Proceedings (Lecture Notes in Computer Science, Vol. 1581)*, Jean-Yves Girard (Ed.). Springer, 228–242. https://doi.org/10.1007/3-540-48959-2_17
- [19] Paul Blain Levy. 2003. Adjunction Models For Call-By-Push-Value With Stacks. *Electronic Notes in Theoretical Computer Science* 69 (2003), 248–271. [https://doi.org/10.1016/S1571-0661\(04\)80568-1](https://doi.org/10.1016/S1571-0661(04)80568-1) CTCS'02, Category Theory and Computer Science.
- [20] Rasmus Ejlers Møgelberg and Marco Paviotti. 2016. Denotational Semantics of Recursive Types in Synthetic Guarded Domain Theory. In *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science* (New York, NY, USA) (LICS '16). Association for Computing Machinery, New York, NY, USA, 317–326. <https://doi.org/10.1145/2933575.2934516>
- [21] Rasmus Ejlers Møgelberg and Niccolò Veltri. 2019. Bisimulation as Path Type for Guarded Recursive Types. *Proc. ACM Program. Lang.* 3, POPL, Article 4 (jan 2019), 29 pages. <https://doi.org/10.1145/3290317>
- [22] E. Moggi. 1989. Computational lambda-calculus and monads. In *[1989] Proceedings. Fourth Annual Symposium on Logic in Computer Science*. 14–23. <https://doi.org/10.1109/LICS.1989.39155>
- [23] Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4) Selections from 1989 IEEE Symposium on Logic in Computer Science.
- [24] H. Nakano. 2000. A modality for recursion. In *Proceedings Fifteenth Annual IEEE Symposium on Logic in Computer Science (Cat. No.99CB36332)*. 255–266. <https://doi.org/10.1109/LICS.2000.855774>
- [25] Georg Neis, Derek Dreyer, and Andreas Rossberg. 2009. Non-Parametric Parametricity. In *International Conference on Functional Programming (ICFP)*. 135–148. <https://doi.org/10.1145/1596550.1596572>
- [26] Max S. New and Amal Ahmed. 2018. Graduality from Embedding-Projection Pairs. *Proc. ACM Program. Lang.* 2, ICFP, Article 73 (jul 2018), 30 pages. <https://doi.org/10.1145/3236768>
- [27] Max S. New, Eric Giovannini, and Daniel R. Licata. 2023. Gradual Typing for Effect Handlers. *Proc. ACM Program. Lang.* 7, OOPSLA2 (2023), 1758–1786. <https://doi.org/10.1145/3622860>
- [28] Max S. New, Dustin Jamner, and Amal Ahmed. 2019. Graduality and Parametricity: Together Again for the First Time. *Proc. ACM Program. Lang.* 4, POPL, Article 46 (dec 2019), 32 pages. <https://doi.org/10.1145/3371114>
- [29] Max S. New and Daniel R. Licata. 2018. Call-by-name Gradual Type Theory. In *Formal Structures for Computation and Deduction, Oxford England*. <https://doi.org/10.4230/LIPIcs.FSCD.2018.24>
- [30] Max S. New, Daniel R. Licata, and Amal Ahmed. 2019. Gradual Type Theory. *Proc. ACM Program. Lang.* 3, POPL, Article 15 (jan 2019), 31 pages. <https://doi.org/10.1145/3290328>
- [31] Michael Shulman. 2007. Framed bicategories and monoidal fibrations. *Theory and Applications of Categories* 20 (06 2007).
- [32] Jeremy G. Siek and Tianyu Chen. 2021. Parameterized cast calculi and reusable meta-theory for gradually typed lambda calculi. *J. Funct. Program.* 31 (2021), e30. <https://doi.org/10.1017/S0956796821000241>
- [33] Jeremy G. Siek and Walid Taha. 2006. Gradual Typing for Functional Languages. In *Scheme and Functional Programming Workshop (Scheme)*. 81–92.
- [34] Jeremy G. Siek, Michael M. Vitousek, Matteo Cimini, and John Tang Boyland. 2015. Refined Criteria for Gradual Typing. In *1st Summit on Advances in Programming Languages (SNAPL 2015) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 32)*, Thomas Ball, Rastislav Bodik, Shriram Krishnamurthi, Benjamin S. Lerner, and Greg Morrisett (Eds.). Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 274–293. <https://doi.org/10.4230/LIPIcs.SNAPL.2015.274>
- [35] Philipp Jan Andries Stassen, Rasmus Ejlers Møgelberg, Maaïke Zwart, Alejandro Aguirre, and Lars Birkedal. 2024. Modelling Probabilistic FPC in Guarded Type Theory. arXiv:2408.04455 [cs.PL] <https://arxiv.org/abs/2408.04455>
- [36] Jonathan Sterling, Daniel Gratzer, and Lars Birkedal. 2022. Denotational semantics of general store and polymorphism. *CoRR* abs/2210.02169 (2022). <https://doi.org/10.48550/ARXIV.2210.02169> arXiv:2210.02169
- [37] Sam Tobin-Hochstadt and Matthias Felleisen. 2006. Interlanguage Migration: From Scripts to Programs. In *Dynamic Languages Symposium (DLS)*. 964–974.
- [38] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The Design and Implementation of Typed Scheme. In *ACM Symposium on Principles of Programming Languages (POPL), San Francisco, California*.

- [39] Niccolò Veltri and Andrea Vezzosi. 2020. Formalizing π -Calculus in Guarded Cubical Agda. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs* (New Orleans, LA, USA) (CPP 2020). Association for Computing Machinery, New York, NY, USA, 270–283. <https://doi.org/10.1145/3372885.3373814>
- [40] Michael M. Vitousek, Cameron Swords, and Jeremy G. Siek. 2017. Big types in little runtime: open-world soundness and collaborative blame for gradual type systems. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages* (Paris, France) (POPL '17). Association for Computing Machinery, New York, NY, USA, 762–774. <https://doi.org/10.1145/3009837.3009849>

A GRADUAL TYPING SYNTAX

THEOREM A.1. For every A , there is a derivation $\text{dyn}(A) : A \sqsubseteq D$

PROOF. By induction on A :

- (1) $A = ?$, then $r(?) : ? \sqsubseteq ?$
- (2) $A = \text{Nat}$, then $\text{Inj}_{\text{Nat}} : \text{Nat} \sqsubseteq ?$
- (3) $A = A_1 \rightarrow A_2$ then $(\text{dyn}(A_1) \rightarrow \text{dyn}(A_2))\text{Inj}_{\rightarrow} : A_1 \rightarrow A_2 \sqsubseteq ?$
- (4) $A = A_1 \times A_2$ then $(\text{dyn}(A_1) \times \text{dyn}(A_2))\text{Inj}_{\times} : A_1 \times A_2 \sqsubseteq ?$

□

THEOREM A.2. For any two derivations $c, c' : A \sqsubseteq A'$ of the same precision $c \equiv c'$

PROOF. (1) We show this by showing that derivations have a canonical form.

The following presentation of precision derivations has unique derivations

$$\begin{array}{c} \text{refl}(D) : D \sqsubseteq D \qquad \text{Inj}_{\text{Nat}} : \text{Nat} \sqsubseteq D \qquad \text{refl}(\text{Nat}) : \text{Nat} \sqsubseteq \text{Nat} \\[10pt] \frac{c : A_i \rightarrow A_o \sqsubseteq D \rightarrow D}{c(\text{Inj}_{\text{arr}}) : A_i \rightarrow A_o \sqsubseteq \text{Nat}} \qquad \frac{c : A_i \sqsubseteq A'_i \quad d : A_o \sqsubseteq A'_o}{c \rightarrow d : A_i \rightarrow A_o \sqsubseteq A'_i \rightarrow A'_o} \end{array}$$

Since it satisfies reflexivity, cut-elimination and congruence, it is a model of the original theory. Since it is a sub-theory of the original theory, it is equivalent.

□

As mentioned in Section 2, the cast calculus separates gradual type casts into downcasts and upcasts, whereas many gradual calculi are built out of a single notion of cast

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash (M :: A') : A'}$$

The term precision rules for this style of calculus are as follows[34]:

$$\begin{array}{c} \frac{\Delta \vdash M \sqsubseteq N_1 : c_1 \quad c_1 : A \sqsubseteq A_1 \quad c_2 : A \sqsubseteq A_2}{\Delta \vdash M \sqsubseteq (N :: A_2) : c_2} \text{CASTRIGHT} \\[10pt] \frac{\Delta \vdash M_1 \sqsubseteq N : c_1 \quad c_1 : A_1 \sqsubseteq A \quad c_2 : A_2 \sqsubseteq A}{\Delta \vdash (M_1 :: A_2) \sqsubseteq N : c_2} \text{CASTLEFT} \end{array}$$

New-Ahmed and New-Licata showed that such if the cast $M_1 :: A_2$ for $M_1 : A_1$ is interpreted as

$$\text{dn}(\text{dyn}(A_2)) \text{ up}(\text{dyn}(A_1)) M$$

then the CastRight and CastLeft rules are derivable from the UpL/UpR/DnL/DnR rules and the *retraction* property of casts: that a downcast after an upcast is equal to the identity. In the intensional setting, this retraction property needs to be weakened: a downcast after an upcast is merely *weakly bisimilar* to the identity. But this weak bisimilarity is no longer strong enough to show the validity of CastRight and CastLeft, due to the lack of transitivity of weak bisimilarity.

Fortunately there is an alternative argument that makes CastRight and CastLeft valid if we change slightly the interpretation of casts. To show this alternative translation we need to first introduce the *least upper bound* of types:

LEMMA A.3. *The preordered set of types with type precision as ordering has all binary least upper bounds defined as follows:*

$$\begin{aligned} (A_1 \times A_2) \sqcup (A'_1 \times A'_2) &= (A_1 \sqcup A'_1) \times (A_2 \sqcup A'_2) \\ (A_1 \rightarrow A_2) \sqcup (A'_1 \rightarrow A'_2) &= (A_1 \rightarrow A'_1) \times (A_2 \rightarrow A'_2) \\ \text{Nat} \sqcup \text{Nat} &= \text{Nat} \\ A \sqcup A' &= ? \text{ otherwise} \end{aligned}$$

Then for $M_1 : A_1$ we translate $M_1 :: A_2$ to

$$\text{dn } c_2^{\sqcup} \text{ up } c_1^{\sqcup} M$$

where $c_1^{\sqcup} : A_1 \sqsubseteq A_1 \sqcup A_2$ and $c_2^{\sqcup} : A_2 \sqsubseteq A_1 \sqcup A_2$. In the presence of the strong retraction property, New and Ahmed showed that this is equal to the prior interpretation. With a weak retraction property, it is still weakly bisimilar to the prior interpretation, so for the extensional properties we care about ultimately this semantics is equivalent. However our compositional arguments do care about this difference, and so we use instead this translation that avoids the need for any retraction:

THEOREM A.4. *If $M_1 :: A_2$ is interpreted as $\text{dn } c_2^{\sqcup} \text{ up } c_1^{\sqcup} M$, then CastRight and CastLeft are admissible.*

PROOF. We show CastLeft, as this is the one that required retraction in prior work. CastRight follows similarly.

Assume $M_1 \sqsubseteq N : c_1$, we seek to prove that

$$\text{dn } c_2^{\sqcup} \text{ up } c_1^{\sqcup} M \sqsubseteq N : c_2$$

Since $A \sqcup A_2$ is the least upper bound, we have there must be a derivation $c^{\sqcup} : A_1 \sqcup A_2 \sqsubseteq A$. Further by Theorem A.2, we have $c_2^{\sqcup} c^{\sqcup} \equiv c_2$, so it suffices to show

$$M_1 \sqsubseteq \text{dn } c_2^{\sqcup} \text{ up } c_1^{\sqcup} N : c_2^{\sqcup} c^{\sqcup}$$

Then applying DnL and UpL it is sufficient to show

$$M_1 \sqsubseteq N : c_1^{\sqcup} c^{\sqcup}$$

But again by Theorem A.2 we have $c_1^{\sqcup} c^{\sqcup} \equiv c_1$ so this follows by our assumption that $M_1 \sqsubseteq N : c_1$. $\square$

B KLEISLI ACTIONS

We now define the Kleisli actions of the type constructors. These are the following

- (1) $- \xrightarrow{k} B : \text{PreDom}_k^{\text{op}} \rightarrow \text{ErrDom}_k$
- (2) $A \xrightarrow{k} - : \text{ErrDom}_k \rightarrow \text{ErrDom}_k$
- (3) $- \times^k A_2 : \text{PreDom}_k \rightarrow \text{PreDom}_k$
- (4) $A_1 \times^k - : \text{PreDom}_k \rightarrow \text{PreDom}_k$

While these can be defined in any CBPV model, for simplicity we just give their definition explicitly for predomains and error domains:

Definition B.1. Given $\phi : L_{\cup} A_i \multimap L_{\cup} A_o$, we define $\phi \xrightarrow{k} B : U(A_o \rightarrow B) \rightarrow U(A_i \rightarrow B)$ as

$$(\phi \xrightarrow{k} B)(f)(x) = f^{\dagger}(\phi(\eta(x)))$$

This is functorial in that $\text{id} \xrightarrow{k} B = \text{id}$ and $(\phi \circ \phi') \xrightarrow{k} B = (\phi' \xrightarrow{k} B) \circ (\phi \xrightarrow{k} B)$.

Further, this preserves squares in that if $\phi \sqsubseteq_{L_{\cup} c_i}^{L_{\cup} c_o} \phi'$ then $\phi \xrightarrow{k} B \sqsubseteq_{U(c_i \rightarrow d)}^{U(c_o \rightarrow d)} \phi' \xrightarrow{k} B$.

PROOF.

- For identity

$$(\text{id} \xrightarrow{k} B)(f)(x) = f^\dagger(\eta(x)) = f(x)$$

- For composition, expanding definitions it suffices to show

$$f^\dagger(\phi(\phi'(\eta(x)))) = ((\phi \xrightarrow{k} B)(f))^\dagger(\phi'(\eta(x)))$$

Which follows if we can show

$$((\phi \xrightarrow{k} B)(f))^\dagger = f^\dagger \circ \phi$$

By the freeness of L_U — it suffices to show it for inputs of the form ηy :

$$((\phi \xrightarrow{k} B)(f))^\dagger(\eta y) = ((\phi \xrightarrow{k} B)(f))(y) = f^\dagger(\phi(\eta y))$$

- For squares, we assume $f \sqsubseteq_{U(c_o \rightarrow d)} f'$ and $x \sqsubseteq_i x'$ and we need to show $f^\dagger(\phi(x)) \sqsubseteq_d (f')^\dagger(\phi'(x))$. This follows easily since $f^\dagger \sqsubseteq_d^{L_U c_o} f'^\dagger$.

□

Definition B.2. Given $f : UB \rightarrow UB'$, define $A \xrightarrow{k} f : U(A \rightarrow B) \rightarrow U(A \rightarrow B')$ as

$$(A \xrightarrow{k} f)(g)(x) = f(g(x))$$

This is functorial in that $A \xrightarrow{k} \text{id} = \text{id}$ and $A \xrightarrow{k} (f \circ f') = (A \xrightarrow{k} f) \circ (A \xrightarrow{k} f')$.

Further this preserves squares in that if $f \sqsubseteq_{U d_o}^{U d_i} f'$ then $A \xrightarrow{k} f \sqsubseteq_{U(c \rightarrow d_o)}^{U(c \rightarrow d_i)} A' \xrightarrow{k} f'$

PROOF. • For identity

$$(A \xrightarrow{k} \text{id})(g)(x) = g(x)$$

- for composition

$$(A \xrightarrow{k} (f \circ f'))(g)(x) = f(f'(g(x))) = (A \xrightarrow{k} f)((A \xrightarrow{k} f')(g))(x)$$

- for squares, given $g \sqsubseteq_{U(c \rightarrow d_i)}^{U(c \rightarrow d_o)} g'$ and $x \sqsubseteq_i x'$ we have $f(g(x)) \sqsubseteq_{d_o} f'(g'(x'))$.

□

Definition B.3 (Kleisli Actions of $\times$). Given $\phi_1 : L_U A_1 \multimap L_U A'_1$ we define $\phi_1 \times^k A_2 : L_U(A_1 \times A_2) \multimap (A'_1 \times A_2)$ as the unique extension of the following to a homomorphism:

$$(\phi_1 \times^k A_2)(\eta(x_1, x_2)) = (\lambda x'_1. \eta(x'_1, x_2))^\dagger(\phi_1(\eta(x_1)))$$

and similarly define

$$(A_1 \times^k \phi_2)(\eta(x_1, x_2)) = (\lambda x'_2. \eta(x_1, x'_2))^\dagger(\phi_2(\eta(x_2)))$$

PROOF. We show cases only for $\multimap \times^k A_2$ as the other is symmetric.

- For identity it is sufficient to consider inputs of the form $\eta(x_1, x_2)$, then indeed

$$(\text{id} \times^k A_2)(\eta(x_1, x_2)) = (\lambda x'_1. \eta(x'_1, x_2))^\dagger(\eta(x_1)) = \eta(x_1, x_2)$$

- For composition, again for pure inputs this reduces to

$$(\lambda x'_1. \eta(x'_1, x_2))^\dagger (\phi'_1(\eta(x_1))) = (\phi'_1 \times^k A_2)((\lambda x'_1. \eta(x'_1, x_2))^\dagger (\phi_1(\eta(x_1))))$$

So it suffices to show

$$(\lambda x'_1. \eta(x'_1, x_2))^\dagger \circ \phi'_1 = (\phi'_1 \times^k A_2) \circ (\lambda x'_1. \eta(x'_1, x_2))^\dagger$$

Since both sides are homomorphisms out of the free error domain by freeness it suffices to show they are equal for inputs of the form $\eta(x'_1)$:

$$(\lambda x'_1. \eta(x'_1, x_2))^\dagger (\phi'_1(\eta(x'_1))) = (\phi'_1 \times^k A_2)(\eta(x'_1, x_2)) = (\phi'_1 \times^k A_2)((\lambda x'_1. \eta(x'_1, x_2))^\dagger (\eta(x'_1)))$$

- Finally, for squares, we need to show if $\phi_l \sqsubseteq_{L_U c'_1}^{L_U c_1} \phi_r$ then $\phi_l \times^k A_{2l} \sqsubseteq_{L_U(c'_1 \times c_2)}^{L_U(c_1 \times c_2)} \phi_r \times^k A_{2r}$.

By the universal property of L_U – on relations, it suffices to assume $x_{1l} c_1 x_{1r}$ and $x_{2l} c_2 x_{2r}$ and prove

$$(\phi_l \times^k A_{2l})(\eta(x_{1l}, x_{2l})) (L_U(c'_1 \times c_2)) (\phi_r \times^k A_{2r})(\eta(x_{1r}, x_{2r}))$$

expanding definitions this is

$$(\lambda x'_{1l}. \eta(x'_{1l}, x_{2l}))^\dagger \phi_l(\eta(x_{1l})) (L_U(c'_1 \times c_2)) (\lambda x'_{1r}. \eta(x'_{1r}, x_{2r}))^\dagger \phi_r(\eta(x_{1r}))$$

By our assumptions we have $\phi_l(\eta(x_{1l})) L_U c'_1 \phi_r(\eta(x_{1r}))$ so since $-^\dagger$ preserves squares it suffices to show

$$(\lambda x'_{1l}. \eta(x'_{1l}, x_{2l})) \sqsubseteq_{L_U(c'_1 \times c_2)}^{L_U c_1} (\lambda x'_{1r}. \eta(x'_{1r}, x_{2r}))$$

To show this assume $x'_{1l} c'_1 x'_{1r}$. Then we have

$$\eta(x'_{1l}, x_{2l}) L_U(c'_1 \times c_2) \eta(x'_{1r}, x_{2l})$$

holds. □

C OMITTED PROOFS SECTION 4

THEOREM C.1. *Let R be a binary relation on the free error domain $U(L_U A)$. If R satisfies the following properties*

- (1) *Transitivity*
- (2) *θ -congruence: If $\triangleright_t(\tilde{x}_t R \tilde{y}_t)$, then $\theta(\tilde{x}) R \theta(\tilde{y})$.*
- (3) *Right step-insensitivity: If $x R y$ then $x R \delta y$.*

Then for any $l : U(L_U A)$, we have $l R \Omega$. If R is left step-insensitive instead then $\Omega R x$.

PROOF. By Löb-induction: we assume that $\triangleright(l R \Omega)$, and we show $(l R \Omega)$. We have $l R \delta l$ by right step-insensitivity. By transitivity, it will therefore suffice to show that $\delta l R \Omega$. Then since $\Omega = \delta \Omega$, it suffices to show $\delta l R \delta \Omega$. By θ -congruence, it suffices to show that $\triangleright_t[(\text{next } l)_t R (\text{next } \Omega)_t]$, i.e., $\triangleright(l R \Omega)$, which is our induction hypothesis.

The proof when R is left step-insensitive is symmetric. □

D DETAILS OF THE RELATIONAL MODEL CONSTRUCTION

In Section 5 we described our model of gradual typing. We omitted several technical proofs involving the well-definedness of composition of relations and the actions of functors. We provide those proofs in this section.

D.1 Free Composition of Error Domain Relations

Definition D.1. Let $d : B_1 \multimap B_2$ and $d' : B_2 \multimap B_3$ be error domain relations. We define the composition dd' inductively by the following rules:

$$\begin{array}{c}
 \frac{b_1 d b_2 \quad b_2 d' b_3}{b_1 d \odot d' b_3} \text{COMP} \qquad \frac{b'_1 \leq_{B_1} b_1 \quad b_1 d \odot d' b_3}{b'_1 d \odot d' b_3} \text{DNCLOSED} \\
 \\
 \frac{b_1 d \odot d' b_3 \quad b_3 \leq_{B_3} b'_3}{b_1 d \odot d' b'_3} \text{UPCLOSED} \qquad \frac{}{\mathcal{U}_{B_1} d \odot d' b_3} \text{PRESErr} \\
 \\
 \frac{\triangleright_t(\tilde{b}_1 d \odot d' \tilde{b}_3)}{\theta_{B_1}(\tilde{b}_1) d \odot d' \theta_{B_3}(\tilde{b}_3)} \text{PRESTHETA}
 \end{array}$$

This has the universal property that $\phi \sqsubseteq_{d''}^{d \odot d'} \phi'$ if and only if for any $b d b'$ and $b' d' b''$ that $\phi(b) d'' \phi'(b'')$.

D.2 The Dynamic Type

The ordering $\leq_D$ on the predomain D is defined by

$$\begin{array}{c}
 \text{Inj}_{\mathbb{N}}(n) \leq \text{Inj}_{\mathbb{N}}(n') \iff n = n' \\
 \text{Inj}_{\times}(d_1, d_2) \leq \text{Inj}_{\times}(d'_1, d'_2) \iff d_1 \leq_D d_2 \text{ and } d'_1 \leq_D d'_2 \\
 \text{Inj}_{\rightarrow}(\tilde{f}) \leq \text{Inj}_{\rightarrow}(\tilde{f}') \iff \triangleright_t(\tilde{f} \leq \tilde{f}')_t,
 \end{array}$$

and the bisimilarity relation is analogous.

D.3 Functorial Actions on Syntactic Perturbations

We provide details on the definition of the homomorphisms specifying the interpretation of perturbations corresponding for the functors L_U , U , $\times$, and $\rightarrow$.

- Then to interpret the perturbations as endomorphisms To define the homomorphism $i_{L_U A}$, by the universal property of the coproduct of monoids it suffices to define two homomorphisms, one from $\mathbb{N} \rightarrow \text{Endo}_{\approx}(L_U A)$ and one from $M_A \rightarrow \text{Endo}_{\approx}(L_U A)$. The first homomorphism sends the generator $1 \in \mathbb{N}$ to the error domain endomorphism $(\delta \circ \eta)^{\dagger}$, and the second sends $m_A \in M_A$ to $L_U(i_A(m_A))$ and we observe in both cases that the resulting endomorphisms are bisimilar to the identity since the action of L_U on morphisms preserves bisimilarity.
- To construct/define the homomorphism $i_{A_1 \times A_2}$, we appeal to the universal property and give homomorphisms $M_{A_1} \rightarrow \text{Endo}_{\approx}(A_1 \times A_2)$ and $M_{A_2} \rightarrow \text{Endo}_{\approx}(A_1 \times A_2)$. The former applies i_{A_1} in the first component and the identity in the second component, while the latter does the reverse with i_{A_2} .
- To construct the interpretation as endomorphisms/define the homomorphism $i_{A \rightarrow B}$ it suffices to define a homomorphism $M_A^{op} \rightarrow \text{Endo}_{\approx}(A \rightarrow B)$ and $M_B \rightarrow \text{Endo}_{\approx}(A \rightarrow B)$. The former takes an element m_A and a predomain morphism $f : A \rightarrow UB$ and returns the composition $f \circ i_A(m_A)$, while the latter is defined similarly by post-composition with i_B .

D.4 Lemmas about Perturbations

The goal of this section is to prove the following lemmas:

LEMMA D.2. Let (A_1, M_{A_1}, i_{A_1}) , (A_2, M_{A_2}, i_{A_2}) , and (A_3, M_{A_3}, i_{A_3}) be value objects, and let $c : A_1 \multimap A_2$ and $c' : A_2 \multimap A_3$ be predomain relations.

Given a push-pull structure Π_c for c and $\Pi_{c'}$ for c' , we can define a push-pull structure $\Pi_{cc'}$ for cc' . Likewise, we can define a push-pull structure for the composition of error domain relations.

PROOF. We define $\Pi_{cc'}^V$ as follows: We first define $\text{push}_{cc'} = \text{push}_{c'} \circ \text{push}_c$ and $\text{pull}_{cc'} = \text{pull}_c \circ \text{pull}_{c'}$. We then observe that the required squares exist for both push and pull. In particular, for push we have that $i_{A_1}(m_{A_1}) \sqsubseteq_c^e i_{A_2}(\text{push}_c(m_{A_1}))$ using the push property for c , and then using the push property for c' we have $i_{A_2}(\text{push}_c(m_{A_1})) \sqsubseteq_{c'}^e i_{A_3}(\text{push}_{c'}(\text{push}_c(m_{A_1})))$. We can then compose these squares horizontally to obtain the desired square. The pull property follows similarly.

The push-pull structure for the composition of computation relations is defined analogously. $\square$

LEMMA D.3. Let A and A' be value objects and let $c : A \multimap A'$ be a relation on the underlying predomains. Given a push-pull structure Π_c for c we can define a push-pull structure $\Pi_{L_U c}$ for $L_U c$ with respect to $L_U A$ and $L_U A'$.

Likewise, let B and B' be computation objects and let $d : B \multimap B'$ be an error domain relation. Given a push-pull structure for d with respect to B and B' , we can define a push-pull structure for Ud with respect to UB and UB' .

PROOF. We define $\text{push}_{L_U c} : \mathbb{N} \oplus M_A \rightarrow \mathbb{N} \oplus M_{A'}$ by the universal property of the coproduct of monoids. In particular, it suffices to define a homomorphism of monoids $\mathbb{N} \rightarrow \mathbb{N} \oplus M_{A'}$ and $M_A \rightarrow \mathbb{N} \oplus M_{A'}$.

The former is simply inl and the latter is $\text{inr} \circ \text{push}_c$.

Then to establish the push property, it suffices by the universal property of the coproduct, and the fact that $\mathbb{N}$ is the free monoid on one generator, to show that the following two squares exist:

$$\begin{array}{ccc} L_U A & \xrightarrow{L_U c} & L_U A' \\ \delta_A^* \downarrow & & \downarrow \delta_{A'}^* \\ L_U A & \xrightarrow{L_U c} & L_U A' \end{array}$$

$$\begin{array}{ccc} L_U A & \xrightarrow{L_U c} & L_U A' \\ L_U(i_A(m_A)) \downarrow & & \downarrow L_U(i_{A'}(\text{push}_c(m_A))) \\ L_U A & \xrightarrow{L_U c} & L_U A' \end{array}$$

where $\delta^* = (\delta \circ \eta)^\dagger$

The first square exists by the fact that the monadic extension operation $-^\dagger$ is monotone in its argument, and the fact that there is a square $\delta_A \circ \eta_A \sqsubseteq \delta_{A'} \circ \eta_{A'}$.

The second square exists by the push property for c and the fact that L_U acts on squares.

The proof for the pull property is dual, and the construction of a push-pull structure for Ud is analogous. $\square$

LEMMA D.4. Let A_1, A'_1, A_2 , and A'_2 be value objects. Let $c_1 : A_1 \multimap A'_1$ and $c_2 : A_2 \multimap A'_2$ be relations on the underlying predomains. Given push-pull structures Π_{c_1} for c_1 and Π_{c_2} for c_2 we can define a push-pull structure $\Pi_{c_1 \times c_2}$ for $c_1 \times c_2$.

PROOF. Recall that the monoid of syntactic perturbations for $A_1 \times A_2$ is $M_{A_1} \oplus M_{A_2}$ and the monoid for $A'_1 \times A'_2$ is $M_{A'_1} \oplus M_{A'_2}$. Thus to define the push homomorphism for $c_1 \times c_2$, it suffices by the universal property of the coproduct of monoids to define a homomorphism $M_{A_1} \rightarrow M_{A'_1} \oplus M_{A'_2}$ and

$M_{A_2} \rightarrow M_{A'_1} \oplus M_{A'_2}$. The former is $\text{inl} \circ \text{push}_{c_1}$, and the latter is $\text{inr} \circ \text{push}_{c_2}$. The pull homomorphism is defined similarly.

For the push property, we need to show that there is a square $i_{A_1 \times A_2}(m_1) \sqsubseteq i_{A'_1 \times A'_2}(\text{push}_{c_1 \times c_2}(m_1))$. By the universal property of the coproduct, it suffices to consider two cases: a perturbation $m_1 \in M_{A_1}$ and a perturbation $m_2 \in M_{A_2}$. In the first case, using the definition of the interpretation homomorphism for the product, the square becomes $(i_{A_1}(m_1) \times \text{id}_{A_2}) \sqsubseteq (i_{A'_1}(\text{push}_{c_1}(m_1)) \times \text{id}_{A'_2})$. Then by the action of $\times$ on squares, it suffices to show that there are squares $i_{A_1}(m_1) \sqsubseteq i_{A'_1}(\text{push}_{c_1}(m_1))$ and $\text{id}_{A_2} \sqsubseteq \text{id}_{A'_2}$. The former is the push property for c_1 , and the latter is an identity square.

The second case, i.e., $m_2 \in M_{A_2}$, is symmetric, and the proof of the pull property is dual. $\square$

LEMMA D.5. *Let A and A' be value objects and B and B' be computation objects. Let $c : A \multimap A'$ be a relation on the underlying predomains and $d : B \multimap B'$ a relation on the underlying error domains. Given push-pull structures Π_c for c and Π_d for d , we can define a push-pull structure $\Pi_{c \rightarrow d}$ for $c \rightarrow d$.*

PROOF. Similar to the product case above. $\square$

We next define the actions of the Kleisli arrow and product functors on syntactic perturbations.

Definition D.6. Let A be a value object and B be a computation object. We define a homomorphism of monoids $\text{id} \xrightarrow{k} - : M_{UB} \rightarrow M_{U(A \rightarrow B)}$ via the universal property of the coproduct of monoids by sending $\mathbb{N}$ to the first injection in $M_{U(A \rightarrow B)} = \mathbb{N} \oplus M_A^{op} \oplus M_B$, and sending M_B to the third injection.

Likewise, we define $- \xrightarrow{k} \text{id} : M_{LUA}^{op} \rightarrow M_{U(A \rightarrow B)}$ as follows. First note that $M_{LUA}^{op} = \mathbb{N} \oplus M_A^{op}$. Then we construct the homomorphism via the universal property, sending $\mathbb{N}$ to the first injection M_A^{op} to the second injection.

We can similarly construct a Kleisli product operation on perturbations:

Definition D.7. Let A_1 and A_2 be value objects. We define a homomorphism of monoids $- \times \text{id} : M_{LUA_1} \rightarrow M_{LUA_1 \times A_2}$ using the universal property, sending $\mathbb{N}$ to the first injection and M_{A_1} to the second injection.

Likewise, we define $\text{id} \times - : M_{LUA_2} \rightarrow M_{LUA_1 \times A_2}$ by sending $\mathbb{N}$ to the first injection and M_{A_2} to the third injection.

It is then easy to verify that the Kleisli arrow action on perturbation is well-defined in that for any $m \in M_{UB}$, we have

$$\text{ptb}_{U(A \rightarrow B)}(\text{id} \xrightarrow{k} m) = \text{id} \xrightarrow{k} i_{UB}(m),$$

where the LHS is the Kleisli action on perturbation and the RHS is the Kleisli action on morphisms. The proof uses the fact that the morphism δ commutes with all error domain morphisms, which is a consequence of the definition of error domain morphism. The other verification involving the Kleisli arrow action is similar, as are the two Kleisli product actions.

D.5 Lemmas involving Quasi-Representable Relations

In this section we prove lemmas about quasi-representability needed for showing that our notions of value and computation relation are compositional, and for defining the action of the functors U , L_U , $\rightarrow$, and $\times$ on value and computation relations.

Recall the notion of quasi-equivalence of relations as defined in Definition 5.8. The following lemma will be useful in showing that two relations are quasi-equivalent.

LEMMA D.8. Let A and A' be value objects, and let $c, c' : A \multimap A'$ be relations between the underlying predomains. If c and c' are both quasi-left-representable by the same predomain morphism $f : A \rightarrow A'$, then $c \approx c'$. If c and c' are both quasi-right-representable by the same predomain morphism $g : A' \rightarrow A$ then $c \approx c'$.

Dually, let B and B' be computation objects, and let $d, d' : B \multimap B'$ be relations between the underlying error domains. If d and d' are both quasi-right-representable by the same error domain morphism $\phi : B' \multimap B$, then $d \approx d'$. If d and d' are both quasi-left-representable by the same $\phi : B \multimap B'$

PROOF. We show the result for c and c' that are quasi-left-representable by the same f ; the other proofs are analogous.

By UpR for c' , there exists a perturbation $\delta_{c'}^{l,e}$ and a square $\text{UpR}_{c'} : \delta_{c'}^{l,e} \sqsubseteq_{c'}^{r(A)} f$.

By UpL for c , there exists a perturbation $\delta_c^{r,e}$ and a square $\text{UpL}_c : f \sqsubseteq_{r(A')}^c \delta_c^{r,e}$.

Composing these horizontally we get the following square:

$$\begin{array}{ccccc} A & \xrightarrow{r(A)} & A & \xrightarrow{c} & A' \\ i_A(\delta_{c'}^{l,e}) \downarrow & & f \downarrow & & \downarrow i_{A'}(\delta_c^{r,e}) \\ A & \xrightarrow{c'} & A' & \xrightarrow{r(A')} & A' \end{array}$$

Then since $r(A)c = c$ and $c'r(A') = c'$, we obtain the desired square. The other square (i.e., with c' on top) is constructed in an analogous manner. $\square$

Next we show that the functors L_U and U preserve quasi-representability.

LEMMA D.9. Let A and A' be value objects and let $c : A \multimap A'$ be a predomain relation. If c is quasi-left-representable, then $L_U c$ is quasi-left-representable. Likewise, if c is quasi-right-representable, then $L_U c$ is quasi-right-representable.

Similarly, let B and B' be computation objects and let $d : B \multimap B'$ be an error domain relation. If d is quasi-right-representable, then Ud is quasi-right-representable.

PROOF. To show that $L_U c$ is quasi-left-representable, we define

- $e_{L_U c} = L_U e_c$
- $\delta_{L_U c}^{r,e} = \text{inr}(\delta_c^{r,e})$ (recalling that the monoid $M_{L_U A'} = \mathbb{N} \oplus M_{A'}$)
- $\delta_{L_U c}^{l,e} = \text{inr}(\delta_c^{l,e})$

We obtain the two squares UpL and UpR using the definition of the interpretation of syntactic perturbations for L_U and the functorial action of L_U on the corresponding squares for c .

The proof for Ud is analogous. $\square$

Next we show that the reflexive relation is quasi-representable.

LEMMA D.10. Let A be a value object. Then $r(A)$ is quasi-left-representable and quasi-right-representable. Likewise, $r(B)$ is quasi-left- and quasi-right-representable for any computation object B .

PROOF. To show $r(A)$ is quasi-left-representable, we take the embedding to be the identity morphism and the perturbations $\delta^{l,e} = \delta^{r,e} = \text{id}_{M_A}$, i.e., the identity of the monoid. Then because i_A is a homomorphism of monoids, it sends the identity element to the identity morphism, so the UpL and UpR squares are just identity squares.

The same argument shows that $r(A)$ is quasi-right-representable and that $r(B)$ is quasi-left- and quasi-right-representable. $\square$

We begin by showing that the composition of quasi-representable relations with push-pull structures is quasi-representable.

LEMMA D.11. *Let $A, A',$ and A'' be value objects, and $B, B',$ and B'' be computation objects. Let $c : A \multimap A'$ and $c' : A' \multimap A''$ be predomain relations with push-pull structures, and let $d : B \multimap B'$ and $d' : B' \multimap B''$ be error domain relations with push-pull structures.*

- (1) *If c and c' are quasi-left- (resp. right)-representable, then cc' is quasi-left- (resp. right)-representable.*
- (2) *If d and d' are quasi-left- (resp. right)-representable, then dd' is quasi-left (resp. right)-representable.*

PROOF.

To show cc' is quasi-left-representable we define

- $e_{cc'} = e_{c'} \circ e_c$
- $\delta_{cc'}^{r,e} = \delta_{c'}^{r,e} \cdot \text{push}_{c'}(\delta_c^{r,e})$ where $\cdot$ denotes multiplication in the monoid M_A
- $\delta_{cc'}^{l,e} = \text{pull}_c(\delta_{c'}^{l,e}) \cdot \delta_c^{l,e}$
- UpL is the following square:

$$\begin{array}{ccccc}
 A & \xrightarrow{c} & A' & \xrightarrow{c'} & A'' \\
 e_c \downarrow & \text{UpL}_c & \downarrow \delta_c^{r,e} & (*) & \downarrow \text{push}_{c'}(\delta_c^{r,e}) \\
 A' & \xrightarrow{r(A')} & A' & \xrightarrow{c'} & A'' \\
 \text{id} \downarrow & & (**) & & \downarrow \text{id} \\
 A' & \xrightarrow{c'} & A'' & & \\
 e_{c'} \downarrow & \text{UpL}_{c'} & \downarrow \delta_{c'}^{r,e} & & \\
 A'' & \xrightarrow{r(A'')} & A'' & &
 \end{array}$$

The square $(*)$ exists by the push-pull property for c' , and the square $(**)$ exists by the downward closure of c' .

- UpR is the following square:

$$\begin{array}{ccccc}
 A & \xrightarrow{r(A)} & A & & \\
 \delta_c^{l,e} \downarrow & \text{UpR}_c & \downarrow e_c & & \\
 A & \xrightarrow{c} & A' & & \\
 \text{id} \downarrow & (*) & \downarrow \text{id} & & \\
 A & \xrightarrow{c} & A' & \xrightarrow{r(A')} & A' \\
 \text{pull}_c(\delta_{c'}^{l,e}) \downarrow & (**) & \delta_{c'}^{l,e} \downarrow & \text{UpR}_{c'} & \downarrow e_{c'} \\
 A & \xrightarrow{c} & A' & \xrightarrow{c'} & A''
 \end{array}$$

To show cc' is quasi-right-representable we define

- $p_{cc'} = p_c \circ p_{c'}$
- $\delta_{cc'}^{l,p} = \delta_c^{l,p} \cdot \text{pull}_c(\delta_{c'}^{l,p})$
- $\delta_{cc'}^{r,p} = \text{push}_{c'}(\delta_c^{r,p}) \cdot \delta_{c'}^{r,p}$

- DNR is the following square:

$$\begin{array}{ccccc}
 A & \xrightarrow{c} & A' & \xrightarrow{c'} & A'' \\
 i_A(\text{pull}_c(\delta_c^{l,p})) \downarrow & & (*) \delta_c^{l,p} \downarrow & \text{DNR}_{c'} \downarrow & p_{c'} \downarrow \\
 A & \xrightarrow{c} & A' & \xrightarrow{r(A')} & A' \\
 \text{id} \downarrow & & & & \downarrow \text{id} \\
 A & \xrightarrow{c} & A' & & \\
 i_A(\delta_d^{l,p}) \downarrow & & \text{DNR}_c \downarrow & & p_c \downarrow \\
 A & \xrightarrow{r(A)} & A & &
 \end{array}$$

Here the square $(*)$ exists by the push-pull property for c .

- DNL is the following square:

$$\begin{array}{ccccc}
 A'' & \xrightarrow{r(A'')} & A'' & & \\
 p_{c'} \downarrow & & \text{DNL}_{c'} \downarrow & & i_{A''}(\delta_{c'}^{r,p}) \downarrow \\
 A' & \xrightarrow{c'} & A'' & & \\
 \text{id} \downarrow & & & & \downarrow \text{id} \\
 A' & \xrightarrow{r(A')} & A' & \xrightarrow{c'} & A'' \\
 p_c \downarrow & \text{DNL}_c \downarrow & \delta_c^{r,p} (*) \downarrow & & i_{A''}(\text{push}_{c'}(\delta_c^{r,p})) \downarrow \\
 A & \xrightarrow{c} & A' & \xrightarrow{c'} & A''
 \end{array}$$

Same as (1), replacing predomains/morphisms/relations/squares with error domains. $\square$

The next two lemmas concern quasi-equivalence and the functors L_U and U .

LEMMA D.12. *Let A, A' , and A'' be value objects and let $c : A \multimap A'$ and $c' : A' \multimap A''$ be predomain relations. Then we have $L_U(cc') \approx L_UcL_Uc'$.*

PROOF. First, we claim that $L_U(cc')$ and L_UcL_Uc' are both quasi-left-represented by $L_Ue_{c'} \circ L_Ue_c$. Indeed, we have by part (1) of Lemma ?? that $e_{c'} \circ e_c$ quasi-left-represents cc' , and then by Lemma ?? we have $L_U(e_{c'} \circ e_c) = L_Ue_{c'} \circ L_Ue_c$ quasi-left-represents $L_U(cc')$. On the other hand, we also know that L_Ue_c quasi-left-represents L_Uc and $L_Ue_{c'}$ quasi-left-represents L_Uc' again by Lemma ?. Then by part (2) of Lemma ??, their composition quasi-left-represents L_UcL_Uc' .

The result now follows by Lemma ?. $\square$

LEMMA D.13. *Let B, B' , and B'' be value objects and let $d : B \multimap B'$ and $d' : B' \multimap B''$ be error domain relations. Then we have $U(dd') \approx UdUd'$.*

PROOF. Analogous to the proof of the previous lemma. $\square$

Now we combine the above lemmas to show a result about the functors L_U and U applied to a composition of relations:

LEMMA D.14. *Let A, A' , and A'' be value objects, and B, B' , and B'' be computation objects. Let $c : A \multimap A'$ and $c' : A' \multimap A''$ be predomain relations with push-pull structures, and let $d : B \multimap B'$ and $d' : B' \multimap B''$ be error domain relations with push-pull structures.*

- (1) *If L_Uc and L_Uc' are quasi-right-representable, then $L_U(cc')$ is quasi-right-representable.*
- (2) *If Ud and Ud' are quasi-left-representable, then $U(dd')$ is quasi-left-representable.*

PROOF. (1) We show that $L_U(cc')$ is quasi-right-representable.

First, by Lemma ?? there is a square α of the form

$$\begin{array}{ccccc} L_U A & \xrightarrow{L_U(cc')} & & L_U A'' & \\ i_{L_U A}(\delta^l) \downarrow & & \alpha & & \downarrow i_{L_U A''}(\delta^r) \\ L_U A & \xrightarrow{L_U c} & L_U A' & \xrightarrow{L_U c'} & L_U A'' \end{array}$$

where δ^l and δ^r are syntactic perturbations.

We define the projection $p_{L_U(cc')}$ to be $p_{L_U c L_U c'} \circ i_{A''}(\delta^r)$. We define $\delta_{L_U(cc')}^{l,p}$ to be $\delta_{L_U c L_U c'}^{l,p} \cdot \delta^l$. Then we can build the DNR square by pasting the square α on top of the DNR square for the composition $L_U c L_U c'$, as shown below:

$$\begin{array}{ccccc} L_U A & \xrightarrow{L_U(cc')} & & L_U A'' & \\ i_{L_U A}(\delta^l) \downarrow & & \alpha & & \downarrow i_{L_U A''}(\delta^r) \\ L_U A & \xrightarrow{L_U c} & L_U A' & \xrightarrow{L_U c'} & L_U A'' \\ i_A(\delta_{L_U c L_U c'}^{l,p}) \downarrow & & \text{DNR}_{L_U c L_U c'} & & \downarrow p_{L_U c L_U c'} \\ L_U A & \xrightarrow{r(L_U A)} & & L_U A & \end{array}$$

We define $\delta_{L_U(cc')}^{r,p}$ to be $\delta_{L_U c L_U c'}^{r,p} \cdot \delta^r$. For DNL, we paste the identity square $\delta^r \sqsubseteq \delta^r$ on top of the DNL square for the composition $L_U c L_U c'$, and below that we paste the square $\text{id} \sqsubseteq_{L_U(cc')}^{L_U c L_U c'}$ which we get from the fact that L_U is lax.

$$\begin{array}{ccccc} L_U A'' & \xrightarrow{r(L_U A'')} & & L_U A'' & \\ i_{A''}(\delta^r) \downarrow & & \text{id} & & \downarrow i_{A''}(\delta^r) \\ L_U A'' & \xrightarrow{r(L_U A'')} & & L_U A'' & \\ p_{L_U c L_U c'} \downarrow & & \text{DNL}_{L_U c L_U c'} & & \downarrow i_{A''}(\delta_{L_U c L_U c'}^{r,p}) \\ L_U A & \xrightarrow{L_U c} & L_U A' & \xrightarrow{L_U c'} & L_U A'' \\ \text{id} \downarrow & & & & \downarrow \text{id} \\ L_U A & \xrightarrow{L_U(cc')} & & L_U A'' & \end{array}$$

(2) The proof that $U(dd')$ is quasi-left-representable is analogous. □

LEMMA D.15. Let $c_1 : A_1 \multimap A'_1$ and $c_2 : A_2 \multimap A'_2$.

- (1) If c_1 and c_2 are quasi-left-representable, then $c_1 \times c_2$ is quasi-left-representable.
- (2) If $L_U c_1$ and $L_U c_2$ are quasi-right-representable, then so is $L_U(c_1 \times c_2)$.

PROOF. (1) To show $c_1 \times c_2$ is quasi-left-representable, we define

- $e_{c_1 \times c_2} = e_{c_1} \times e_{c_2}$
- $\delta_{c_1 \times c_2}^{r,e} = \text{inl}(\delta_{c_1}^{r,e}) \cdot \text{inr}(\delta_{c_2}^{r,e})$ (recall from Section 5.3.1 that $M_{A'_1 \times A'_2}$ is the coproduct $M_{A'_1} \oplus M_{A'_2}$)
- $\delta_{c_1 \times c_2}^{l,e}$ is defined similarly

- The UpL square is constructed using the functorial action of $\times$ and the corresponding squares for c_1 and c_2 :

$$\begin{array}{ccccc}
 A_1 \times A_2 & \xrightarrow{c_1 \times c_2} & A'_1 \times A'_2 & & \\
 e_{c_1} \times \text{id} \downarrow & \text{UpL}_{c_1} \times \text{id} & \downarrow i_{A'_1}(\delta_{c_1}^{r,e}) \times \text{id} & & \\
 A'_1 \times A_2 & \xrightarrow{r(A'_1) \times c_2} & A'_1 \times A'_2 & & \\
 \text{id} \times e_{c_2} \downarrow & \text{id} \times \text{UpL}_{c_2} & \downarrow \text{id} \times i_{A'_2}(\delta_{c_2}^{r,e}) & & \\
 A'_1 \times A'_2 & \xrightarrow{r(A'_1) \times r(A'_2)} & A'_1 \times A'_2 & &
 \end{array}$$

- The UpR square is defined similarly

(2) To show that $L_U(c_1 \times c_2)$ is quasi-right-representable, we define

- $p_{L_U(c_1 \times c_2)} = (p_{L_U c_1} \overset{k}{\times} A_2) \circ (A'_1 \overset{k}{\times} p_{L_U c_2})$
- $\delta_{L_U(c_1 \times c_2)}^{l,p} = (\delta_{L_U c_1}^{l,p} \overset{k}{\times} \text{id}) \cdot (\text{id} \overset{k}{\times} \delta_{L_U c_2}^{l,p})$ using the Kleisli product actions on syntactic perturbations in Definition ??
- $\delta_{L_U(c_1 \times c_2)}^{r,p} = (\delta_{L_U c_1}^{r,p} \overset{k}{\times} \text{id}) \cdot (\text{id} \overset{k}{\times} \delta_{L_U c_2}^{r,p})$
- The squares are obtained via the functorial action of $\overset{k}{\times}$ on the squares for $L_U c_1$ and $L_U c_2$.

□

LEMMA D.16. Let $c : A \multimap A'$ and $d : B \multimap B'$.

- (1) If c is quasi-left-representable and d is quasi-right-representable, then $c \rightarrow d$ is quasi-right-representable.
- (2) If $L_U c$ is quasi-right-representable and Ud is quasi-left-representable, then $U(c \rightarrow d)$ is quasi-left-representable.

PROOF. (1) To show $c \rightarrow d$ is quasi-right-representable, we define:

- $p_{c \rightarrow d} = e_c \rightarrow p_d$
- $\delta_{c \rightarrow d}^{l,p} = \text{inl}(\delta_c^{l,e}) \cdot \text{inr}(\delta_d^{l,p})$
- $\delta_{c \rightarrow d}^{r,p} = \text{inl}(\delta_c^{r,e}) \cdot \text{inr}(\delta_d^{r,p})$
- The squares DNR and DNL are obtained via the functorial action of $\rightarrow$ on squares.

(2) We show $U(c \rightarrow d)$ is quasi-left-representable as follows:

- $e_{U(c \rightarrow d)} = (p_{L_U c} \overset{k}{\rightarrow} B') \circ (A \overset{k}{\rightarrow} e_{Ud})$
- $\delta_{U(c \rightarrow d)}^{r,e} = (\delta_{L_U c}^{r,p} \overset{k}{\rightarrow} B') \cdot (A' \overset{k}{\rightarrow} \delta_{Ud}^{r,e})$ using the Kleisli arrow actions on syntactic perturbations in Definition ??.
- $\delta_{U(c \rightarrow d)}^{l,e} = (\delta_{L_U c}^{l,p} \overset{k}{\rightarrow} B) \cdot (A \overset{k}{\rightarrow} \delta_{Ud}^{l,e})$
- The squares UpL and UpR are obtained via the functorial action of $\overset{k}{\rightarrow}$ on squares. For instance, UpL is given by the following square:

$$\begin{array}{ccc}
U(A \rightarrow B) & \xrightarrow{U(c \rightarrow d)} & U(A' \rightarrow B') \\
\downarrow A \xrightarrow{k} e_{Ud} & \text{id}_{Fc} \xrightarrow{k} \text{UpL}_{Ud} & \downarrow A' \xrightarrow{k} \delta_{Ud}^{r,e} \\
U(A \rightarrow B') & \xrightarrow{U(c \rightarrow r(B'))} & U(A' \rightarrow B') \\
\downarrow p_{Fc} \xrightarrow{k} B' & \text{DNL}_{Fc} \xrightarrow{k} \text{id}_{r(B')} & \downarrow \delta_{Fc}^{r,p} \xrightarrow{k} B' \\
U(A' \rightarrow B') & \xrightarrow{U(r(A') \rightarrow r(B'))} & U(A' \rightarrow B')
\end{array}$$

The construction of UpR is similar.

□

D.6 Model Construction: Composition and Functorial Actions on Relations

We now show that our notions of value and computation relations compose.

Definition D.17 (composition of value (resp. computation) relations). Let A, A' and A'' be value objects and let c and c' be *value relations* between A and A' and A' and A'' respectively. Then we define their composition cc' as follows:

- The predomain relation is the composition of the underlying predomain relations cc'
- The push-pull structure is given by Lemma ??.
- Quasi-left-representability of cc' follows from Lemma ?? and the fact that c and c' are quasi-left-representable
- Quasi-right-representability of $L_U(cc')$ holds by Lemma ?? and the quasi-right-representability of L_Uc and L_Uc'

Likewise, let B, B' and B'' be computation objects and let d and d' be *computation relations* between B and B' and B' and B'' respectively. We define their composition dd' as follows:

- The error domain relation is the composition of the underlying error domain relations dd'
- The push-pull structure is given by Lemma ??.
- Quasi-right-representability of dd' follows from Lemma ?? and the fact that d and d' are quasi-right-representable
- Quasi-left-representability of $U(dd')$ holds by Lemma ?? and the quasi-left-representability of Ud and Ud'

We can define the functorial actions of $L_U, U, \times$, and $\rightarrow$ on value and computation relations as follows:

Definition D.18 (functorial actions on relations). (1) Let A and A' be value objects and $c : A \multimap A'$ a value relation. Then we define the computation relation L_Uc as follows:

- The error domain relation is given by the action of L_U on the underlying predomain relation of c
- The push-pull structure is given by Lemma ??
- Quasi-right-representability of L_Uc holds because it is part of the definition of the value relation c
- Quasi-left-representability of UL_Uc follows from Lemma ?? and the quasi-left-representability of c

(2) Let B and B' be computation objects and $d : B \multimap B'$ a computation relation. We define Ud as follows:

- The predomain relation is given by the action of U on the error domain relation of d

- The push-pull structure is given by Lemma ??
 - Quasi-left-representability of Ud holds because it is part of the definition of the computation relation d
 - Quasi-right-representability of $L_U(Ud)$ follows from Lemma ?? and the quasi-right-representability of d
- (3) Let $c_1 : A_1 \multimap A'_1$ and $c_2 : A_2 \multimap A'_2$ be value relations. We define $c_1 \times c_2$ as follows:
- The relation is given by the action of $\times$ on the predomain relations of c_1 and c_2
 - The push-pull structure is given by Lemma ??
 - Quasi-left-representability of $c_1 \times c_2$ and quasi-right-representability of $L_U(c_1 \times c_2)$ are given by Lemma ??
- (4) Let $c : A \multimap A'$ be a value relation and $d : B \multimap B'$ be a computation relation. We define $c \rightarrow d$ as follows:
- The error domain relation is given by the action of $\rightarrow$ on the underlying relations of c and d
 - The push-pull structure is given by Lemma ??
 - Quasi-right-representability of $c \rightarrow d$, and quasi-left-representability of $U(c \rightarrow d)$ follow from Lemma ??

Lastly, we prove quasi-equivalence of relations involving composition and the functors U , L_U , $\rightarrow$ and $\times$.

LEMMA D.19. *The following hold:*

- $U(dd') \approx U(d)U(d')$
- $L_U(cc') \approx L_U(c)L_U(c')$
- $(cc') \rightarrow (dd') \approx (c \rightarrow d)(c' \rightarrow d')$
- $(c_1c'_1) \times (c_2c'_2) \approx (c_1 \times c_2)(c'_1 \times c'_2)$

PROOF. (1) and (2) were already shown in Lemmas ?? and ??. (3) follows from observing that both relations are right-represented by the morphism $e'_c e_c \rightarrow p_d p_{d'}$ and (4) from the fact that both are left-represented by the morphism $e_{c'_1} e_{c_1} \times e_{c'_2} e_{c_2}$ $\square$

D.7 Definitions of Error Ordering and Weak Bisimilarity for the Delay Monad

We define a notion of lock-step error ordering and weak bisimilarity relation for the coinductive delay monad $\text{Delay}(\mathbb{N} + \mathbb{U})$:

The lock-step error ordering is defined coinductively by the following rules:

$$\frac{}{\text{now}(\text{inr } 1) \leq^{\text{Del}} d} \quad \frac{x_1 \leq_X x_2}{\text{now}(\text{inl } x_1) \leq^{\text{Del}} \text{now}(\text{inl } x_2)} \quad \frac{d_1 \leq^{\text{Del}} d_2}{\text{later } d_1 \leq^{\text{Del}} \text{later } d_2}$$

And we similarly define by coinduction a “weak bisimilarity” relation on $\text{Delay}(\mathbb{N} + \mathbb{U})$. This uses a relation $d \Downarrow x?$ between $\text{Delay}(\mathbb{N} + \mathbb{U})$ and $\mathbb{N} + \mathbb{U}$ that is defined as $d \Downarrow n? := \sum_{i \in \mathbb{N}} d = \text{later}^i(\text{now } n?)$. Then weak bisimilarity for the delay monad is defined coinductively by the rules

$$\frac{n? \approx_{\mathbb{N} + \mathbb{U}} m?}{\text{now } n? \approx^{\text{Del}} \text{now } m?} \quad \frac{d_1 \Downarrow n? \quad n? \approx_{\mathbb{N} + \mathbb{U}} m?}{\text{later } d_1 \approx^{\text{Del}} \text{now } m?} \quad \frac{d_2 \Downarrow m? \quad n? \approx_{\mathbb{N} + \mathbb{U}} m?}{\text{now } n? \approx^{\text{Del}} \text{later } d_2} \quad \frac{d_1 \approx^{\text{Del}} d_2}{\text{later } d_1 \approx^{\text{Del}} \text{later } d_2}$$

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because $\text{\LaTeX}$ now knows how many pages to expect for this document.